Scopus AI Offline

Presentation guide: architecture and workflow of the embedding app and the search app

Embedding app → Bundle → Search app

This guide holds all 40 slides of the presentation. Each page shows one slide with the full script to say while it is on screen. Blue always means the embedding app, and orange always means the search app.

Timing

Section	Slides	Time
Overview	1–5	6 min
Part 1: Embedding app	6–18	17 min
Part 2: Search app	19–33	21 min
Part 3: Deployment	34–38	5 min
Talk in total		**49 min**
Questions (prepared answers on slide 39)		10 min

Estimated from the script length at 130 words per minute, plus 8 seconds per slide. Practise once with a timer and adjust.

Shorter version

For about 38 minutes, skip slides 10, 15, 18, 23, 26, 30, 32 and 36. Those slides hold detail, and the story still works without them.

How to prepare

- Read the scripts aloud a few times, then speak in your own words.
- On the diagrams, point at each box as you name it.
- Keep slide 39 ready for the question round. Its script holds the fuller answers.
- Do not project slide 40. It is for you.

Optional live demo

Test it first, with the apps running and the models loaded, because the first question after a start can be slow.

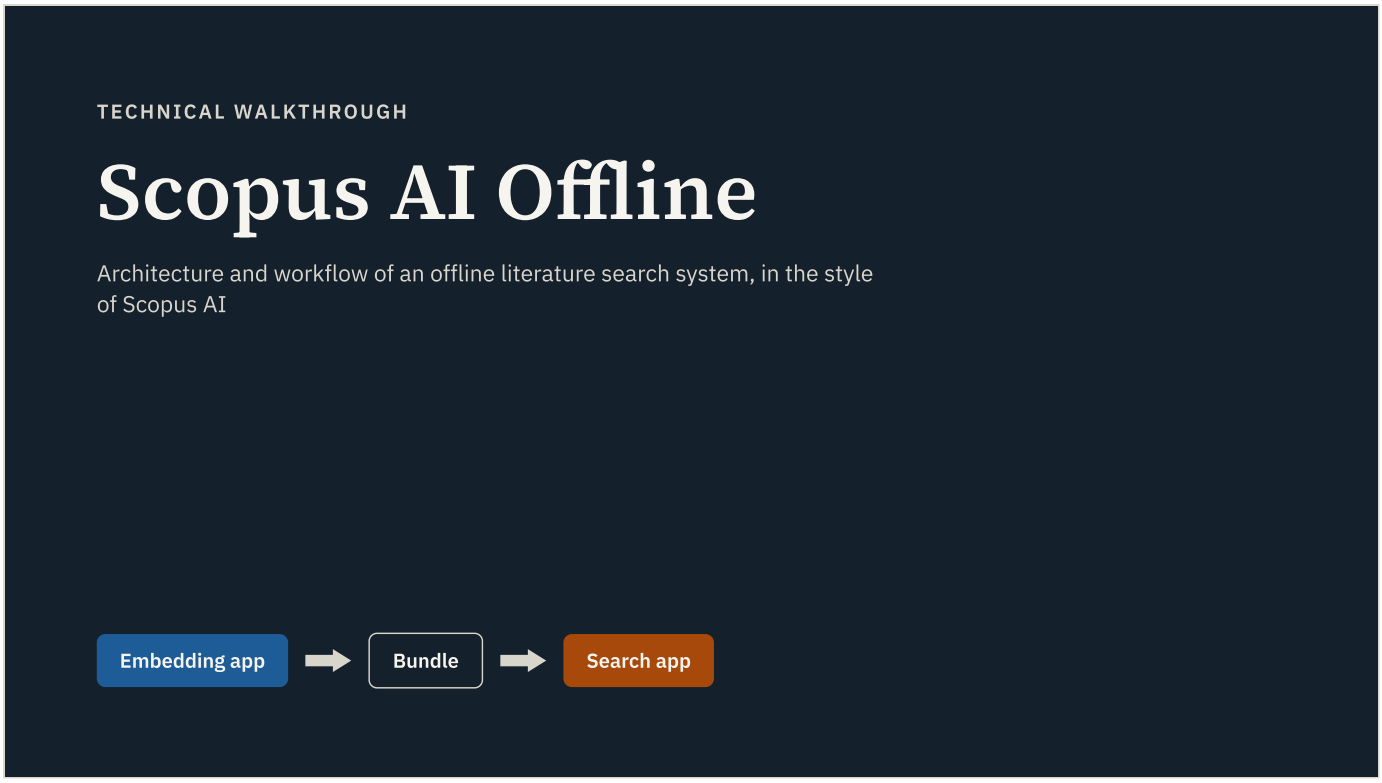
1. Ask a research question and open a citation.
2. Ask a follow-up and show “Searched for”.
3. Ask something off-topic and show the refusal.
4. Open /admin and show the Activity tab.

Slide list

#	Title	Section	Time
1	Scopus AI Offline	Overview	1 min
2	What the System Does	Overview	1 min 10 s
3	Presentation Plan	Overview	1 min
4	Two Applications, One Hand-off	Overview	1 min 30 s
5	What the Two Apps Must Agree On	Overview	1 min 20 s
6	The Embedding App	Part 1	40 s
7	Embedding App Architecture	Part 1	1 min 30 s
8	Ingest Workflow: Seven Stages	Part 1	1 min 10 s
9	Reading the Source Data	Part 1	1 min 20 s
10	From Rows to Papers	Part 1	1 min 20 s
11	Chunking a Paper	Part 1	1 min 10 s
12	One Point, Two Vectors	Part 1	1 min 30 s
13	Streaming Batches	Part 1	1 min 20 s
14	Two Ingest Modes	Part 1	1 min 20 s
15	Failure Handling	Part 1	1 min 20 s
16	Quality Check	Part 1	1 min 20 s
17	Export Bundle	Part 1	1 min 20 s
18	Embedding Performance	Part 1	1 min 20 s
19	The Search App	Part 2	40 s
20	Search App Architecture	Part 2	1 min 10 s
21	Importing a Bundle	Part 2	1 min 40 s
22	Life of a Question	Part 2	1 min 40 s
23	Follow-up Questions	Part 2	1 min 30 s
24	Question Guard	Part 2	1 min 30 s
25	Hybrid Retrieval	Part 2	1 min 40 s
26	Relevance Gates	Part 2	1 min 10 s
27	Grounded Generation	Part 2	1 min 20 s
28	Sentence Verification	Part 2	1 min 30 s
29	Streaming Path	Part 2	1 min 30 s
30	What the User Receives	Part 2	1 min 10 s
31	Grounding Test Results	Part 2	1 min 20 s
32	Accounts and Administration	Part 2	1 min 20 s
33	Capacity on a 6 GB GPU	Part 2	1 min 30 s
34	Deployment and Operation	Part 3	20 s
35	Building the Offline Packages	Part 3	1 min 20 s
36	Updating the Index	Part 3	1 min 10 s
37	Known Limits	Part 3	1 min 30 s
38	Summary	Part 3	1 min
39	Prepared Answers	Appendix	1 min 50 s
40	Presenter Guide (Not for Projection)	Appendix	1 min

Slide 1 Overview

about 1 min



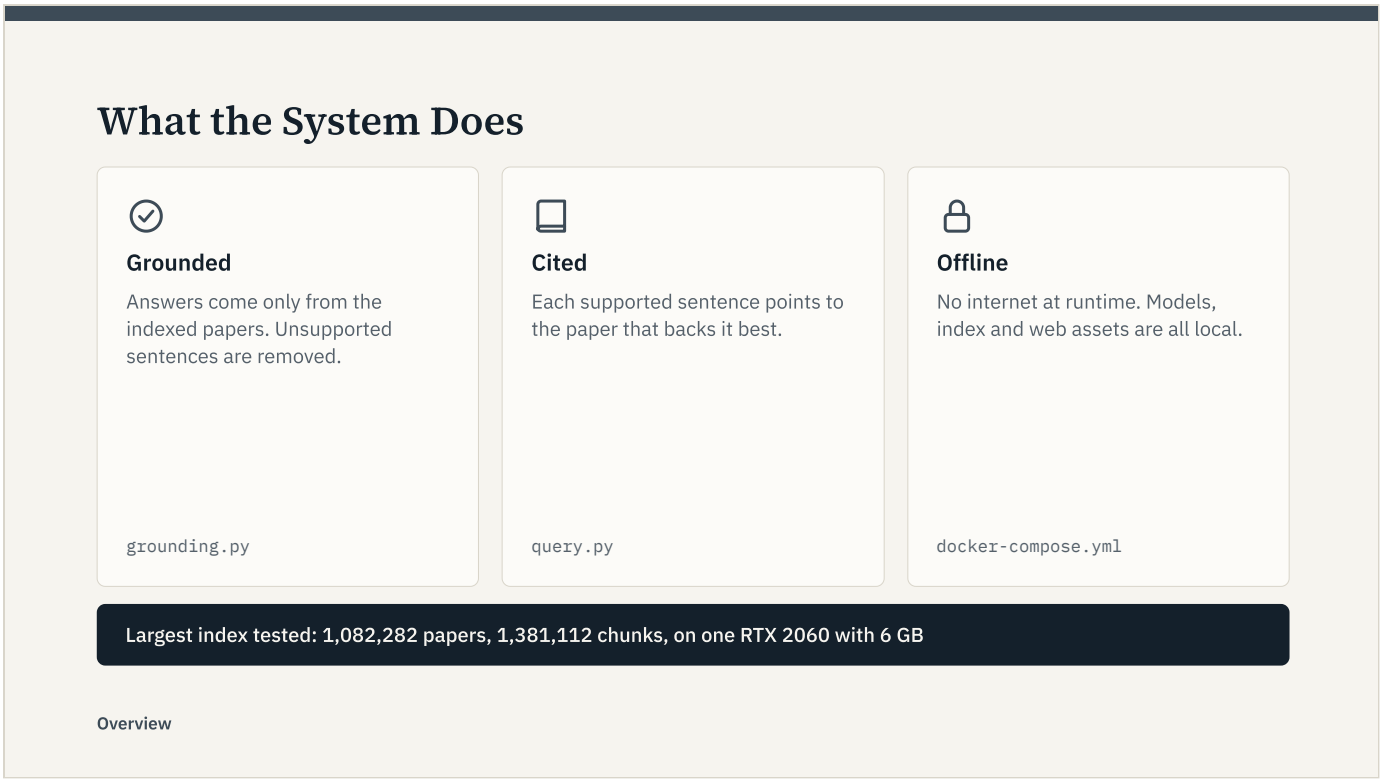
SCRIPT

Hello everyone, and thank you for joining. Today I will explain the full architecture and workflow of our offline literature search system.

We call it Scopus AI Offline, because it works in the style of Scopus AI. You ask a research question, and you get a written answer with citations. The difference is that everything here runs locally, on our own PCs, with no internet connection.

I will explain the system in two separate parts. First the embedding app, which builds the search index. Then the search app, which answers questions. After that I will show how both apps reach offline PCs.

Short questions are welcome at any time. Please keep longer questions for the end.



SCRIPT

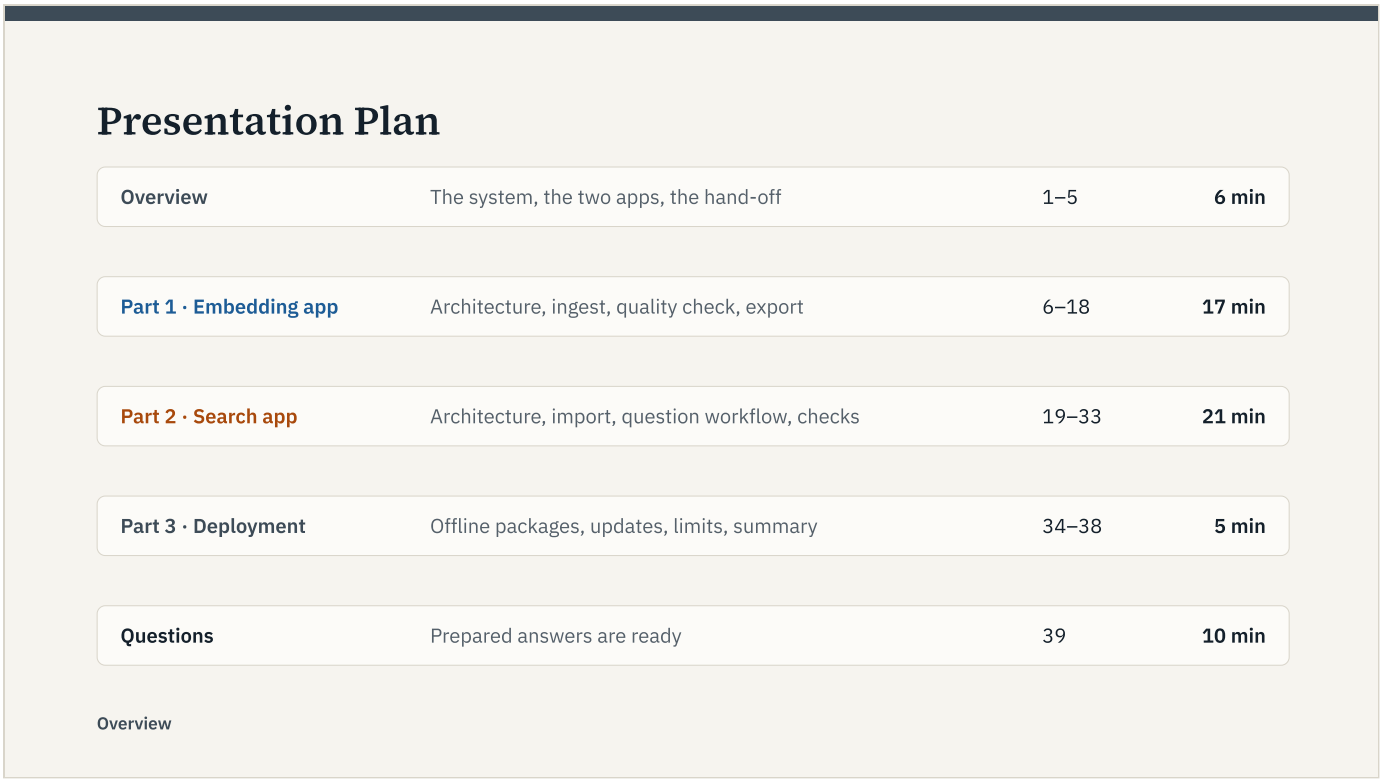
Let me start with what the system does, in plain terms. A researcher types a question on a chat page. The system finds the most relevant papers in a local index, and writes an answer from those papers only.

It makes three promises. First, grounded: the answer contains only what the retrieved papers say. A sentence that the papers do not support is removed before the user sees it.

Second, cited: each supported sentence points to the paper that supports it best.

Third, offline: nothing is downloaded at runtime. The models, the index and even the web page assets are local.

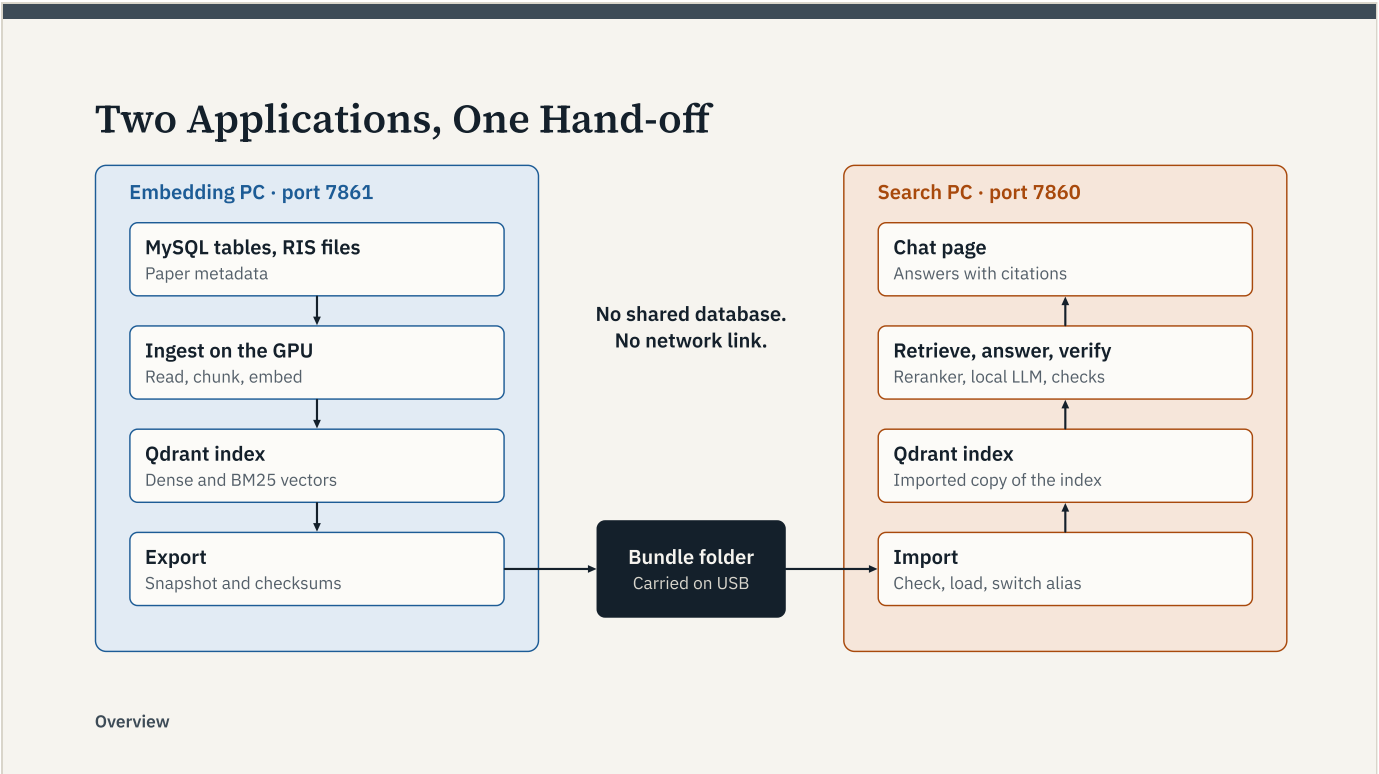
The line at the bottom shows the largest index we have tested: about 1.08 million papers, which became about 1.38 million chunks, on a single RTX 2060 with 6 GB of memory. So it works at real scale on modest hardware.



SCRIPT

Here is the plan. I will start with a short overview of the whole system. Part 1 is the embedding app: how the index is built, tested and exported. Part 2 is the search app, which is the longest part, because it holds most of the logic: how a question becomes a checked answer. Part 3 is short and covers deployment on offline PCs, the known limits and a summary.

The colours will help you follow along. Blue always means the embedding app. Orange always means the search app. I will keep questions for the end, and I have prepared some answers.



SCRIPT

This is the whole system on one slide. There are two separate applications on two separate PCs.

On the left, in blue, is the embedding PC. Paper metadata comes in as MySQL tables or RIS files. The GPU embeds it, the result goes into a Qdrant index, and we export that index.

In the middle is the only link between the two: a bundle folder, carried on a USB drive.

On the right, in orange, is the search PC. It imports the bundle into its own Qdrant, and then answers questions on a chat page. So follow the arrows: down the left side, across, and up the right side.

Notice what is missing. The two apps share no database and no network connection. Each one has its own Docker images, its own accounts and its own data.

This split has a practical benefit. Building an index is a long GPU job, more than an hour for a million papers. Answering questions also needs the GPU. With two PCs, a long ingest never slows down the users.

What the Two Apps Must Agree On

What must match	Current value	How it is enforced
Index layout	scopus-papers-v2	paper_index.py is identical in both apps; import compares INDEX_FORMAT
Embedding model	multilingual-e5-small, 384 dimensions	embedding.py is identical; import compares name and vector size
Python packages	Same PyTorch and sentence-transformers	Embedding image copies the search image's packages
Qdrant version	1.12.6 in both apps	A bundle from a newer Qdrant is refused
File integrity	SHA-256 for every file	Checked before anything is loaded

Overview app/core/paper_index.py

SCRIPT

Two apps with no connection still have to agree on a few things, or the search results would be wrong. This table lists all of them.

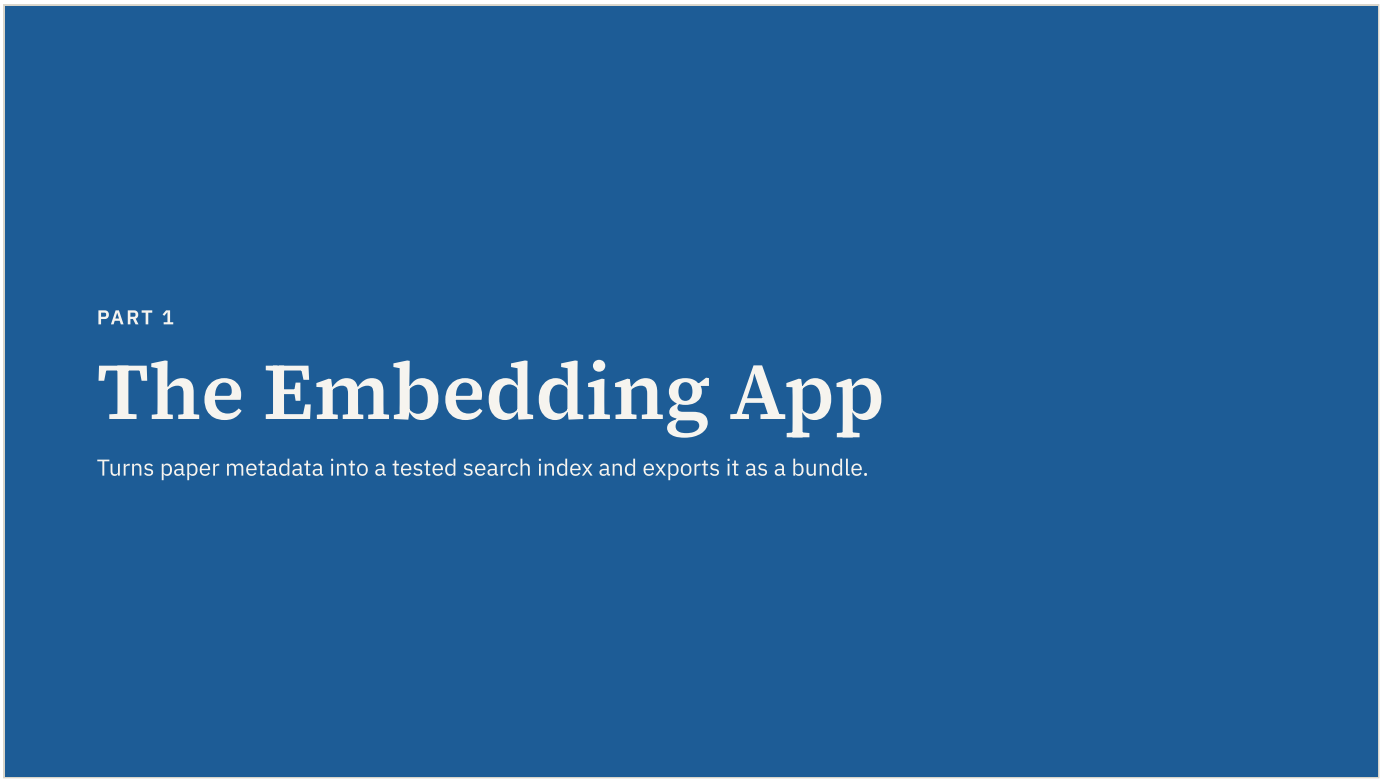
The index layout has a name: scopus-papers-v2. The file paper_index.py defines it, and the same file exists in both apps. If the layout ever changes, we raise that name, and the search app refuses old bundles instead of searching them wrongly.

The embedding model must be the same, because questions and papers must live in the same vector space. The import compares the model name and the vector size.

The Python packages also match, because the embedding image is built from the search image. So both apps compute the same vectors.

A bundle cannot come from a newer Qdrant than the one on the search PC. And every file has a SHA-256 checksum.

The key point: all these checks happen during the import, before anything changes on the search PC.

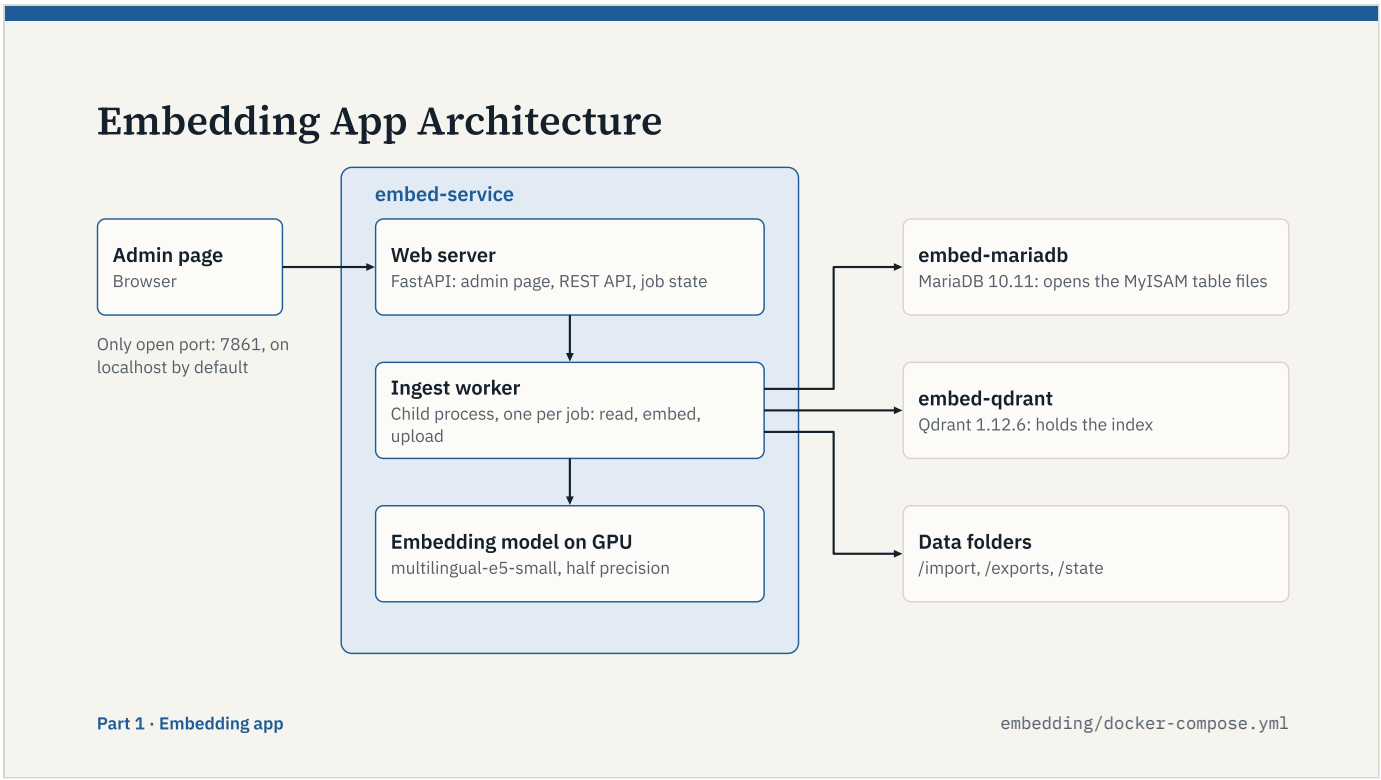


SCRIPT

Now Part 1, the embedding app. Its job is easy to state: take paper metadata, turn it into a searchable index, test it, and export it as a bundle.

It has no chat and no language model. It only has an administrator web page on port 7861.

I will show the architecture first. Then I will follow the data through the workflow, in the same order the data moves.



SCRIPT

Here is how the embedding app is built. It is three Docker containers.

The big blue box is embed-service, which is our Python code. Inside it there are two processes. The web server is a FastAPI app. It serves the admin page, receives commands and keeps the job state.

When an ingest starts, the server spawns a separate child process: the ingest worker. This is an important design choice. All the heavy work and all the GPU use happen in the worker. When the job ends, the process exits, so all GPU memory goes back to the system, and the server stays small. The worker sends progress events back through a queue, and the page polls for status every second.

On the right are the supporting services. Qdrant holds the index. MariaDB is here for one reason: it reads the MySQL table files during an ingest. Neither of them is published outside Docker.

The only open port is 7861, and by default only on localhost. Only administrators can use this app.

Ingest Workflow: Seven Stages

1 2 3 **4 · Read, embed and upload** 5 6 7

#	Stage	What happens	Weight
1	Scan folder	Find complete tables and RIS files	1
2	Read the current index	Collect keys of indexed papers (Add mode only)	3
3	Load MySQL tables	Copy table files into MariaDB staging	4
4	Read, embed and upload	Stream batches of 1,000 papers into Qdrant	83
5	Optimize vector index	Wait until Qdrant finishes the HNSW graph	5
6	Check search quality	Look up a sample of new papers by title	2
7	Publish index	Switch alias, write index_meta.json	2

Part 1 · Embedding app `embedding_service/pipeline.py`

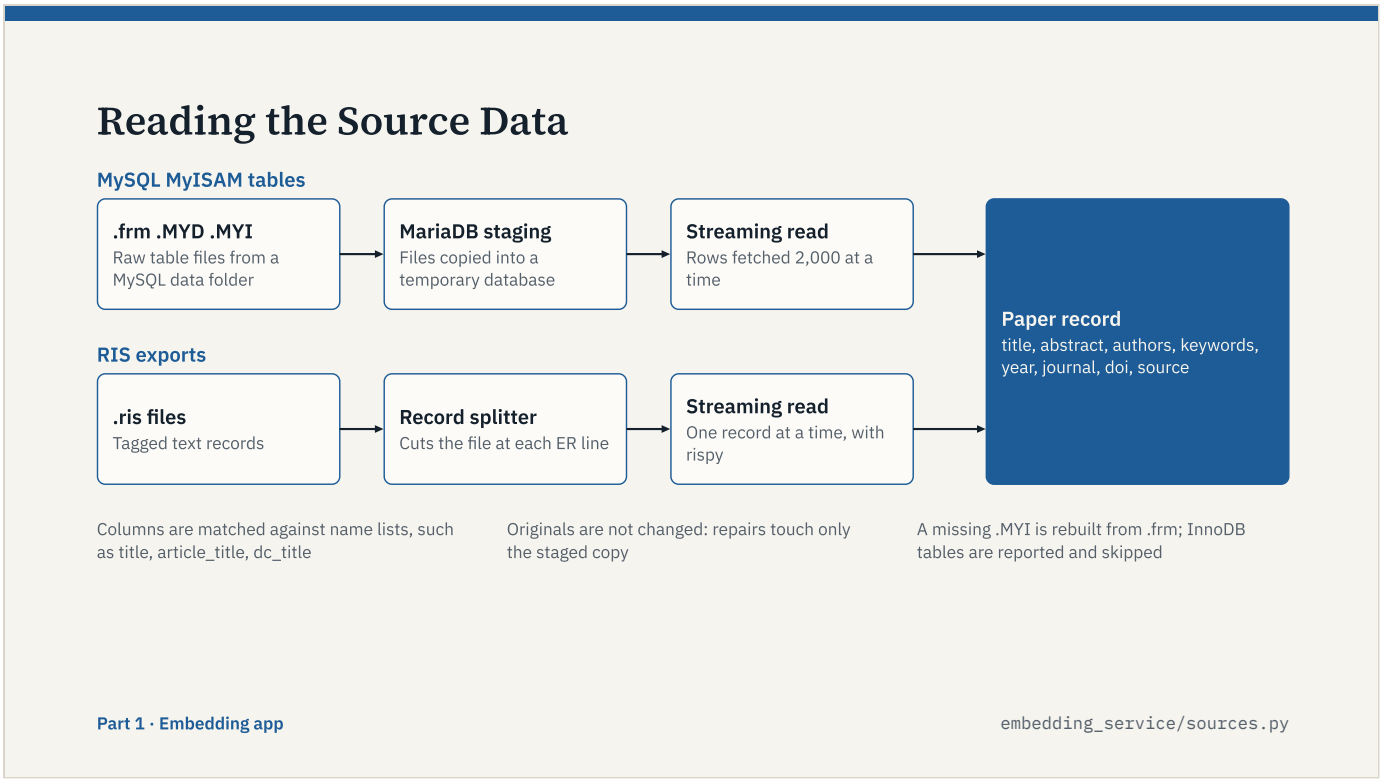
SCRIPT

An ingest always runs through these seven stages. The bar at the top shows their weights in the progress bar. These are fixed weights in the code, not measured times, but they do show where the time goes. Stage 4 alone is 83 percent.

Stage 1 scans the folder for complete tables and RIS files. Stage 2 runs only when we add to an existing index: it reads the keys of all indexed papers, so that we can skip duplicates. Stage 3 copies the MySQL table files into MariaDB.

Stage 4 is the main work: read, embed and upload, in batches. Stage 5 waits for Qdrant to finish its vector index. Stage 6 tests search quality on a sample. Stage 7 publishes the result.

The next slides follow these stages in order, starting with how the data is read.



SCRIPT

The app accepts two input types, and both end as the same paper record.

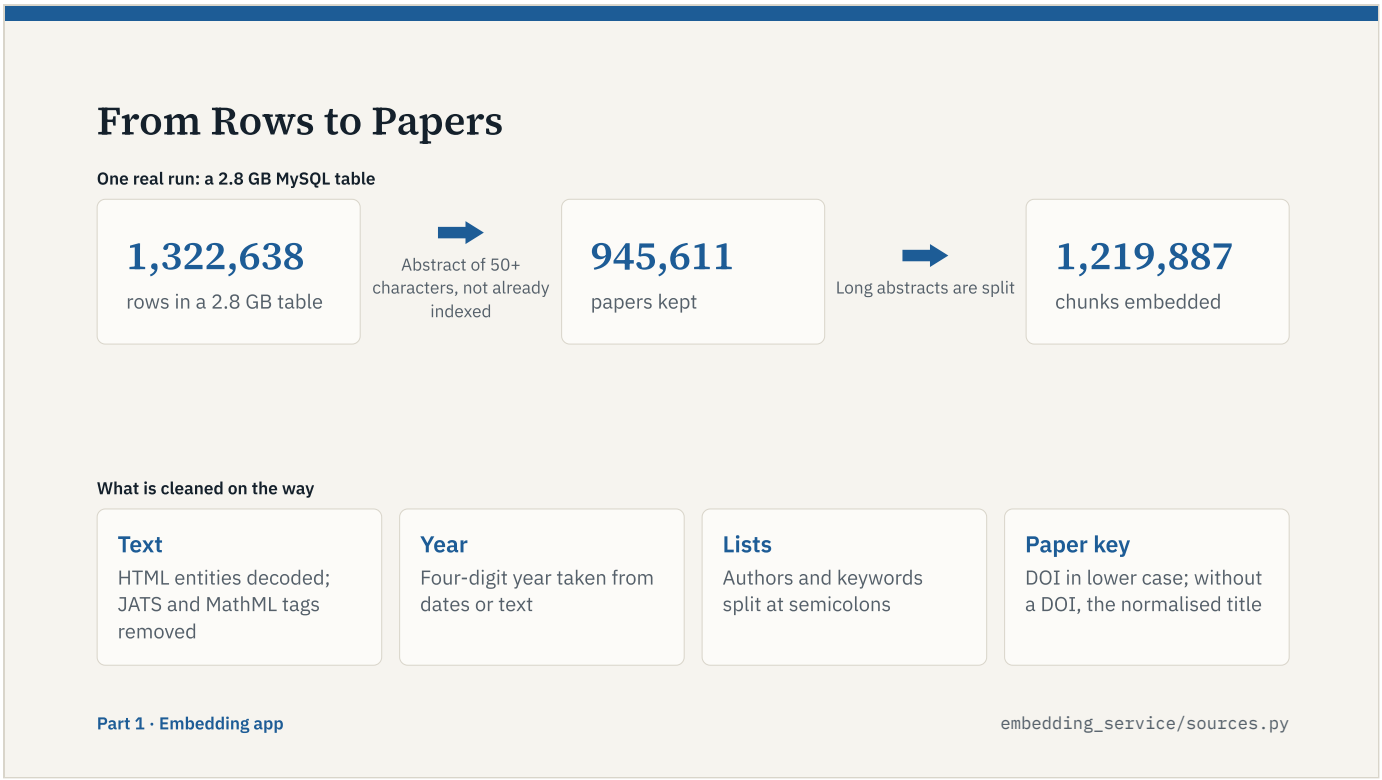
The top lane is MySQL. We receive raw MyISAM table files: .frm, .MYD and .MYI. Python cannot read those directly. So the worker creates a temporary staging database in MariaDB, copies the files into its data folder, and lets MariaDB open them. Then it reads rows with a streaming cursor, so a huge table never sits in memory.

The originals are not changed. If a table is marked as crashed, only the staged copy is repaired. If the .MYI file is missing, it is rebuilt from the .frm file.

The bottom lane is RIS. RIS files are read one record at a time, cutting the file at each ER line.

Column names differ between databases, so the reader matches them against a list of candidates, for example title, article_title or dc_title. A table with no abstract column is skipped with a warning.

Both lanes produce the same record, shown on the right.



SCRIPT

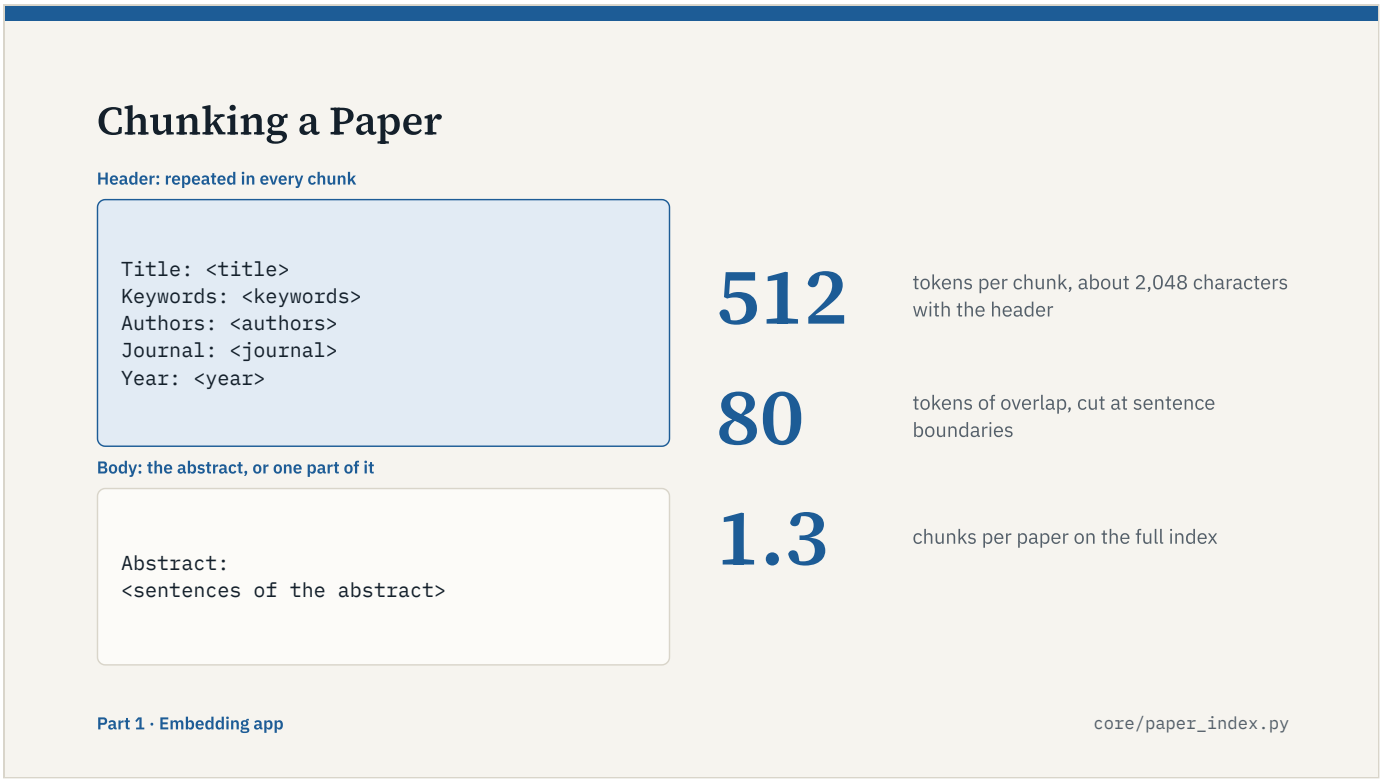
Not every row becomes a paper. This example is a real run: a 2.8 gigabyte MySQL table with about 1.32 million rows.

Two filters apply. First, a row needs an abstract of at least 50 characters, because the abstract is what we search and answer from. Second, a paper that is already indexed is skipped.

After the filters, about 946 thousand papers remained. These became about 1.22 million chunks, because long abstracts are split.

The cards at the bottom show the cleaning. Text has its HTML entities decoded, and JATS and MathML tags removed. Years are reduced to four digits. Authors and keywords are split at semicolons.

The most important one is the paper key. It is the DOI in lower case. If there is no DOI, it is the normalised title. This key is how the system knows that two records are the same paper. These are small steps, but they matter, because this text is what the language model reads later.



SCRIPT

Next, each paper becomes one or more chunks. A chunk is the unit we embed and search.

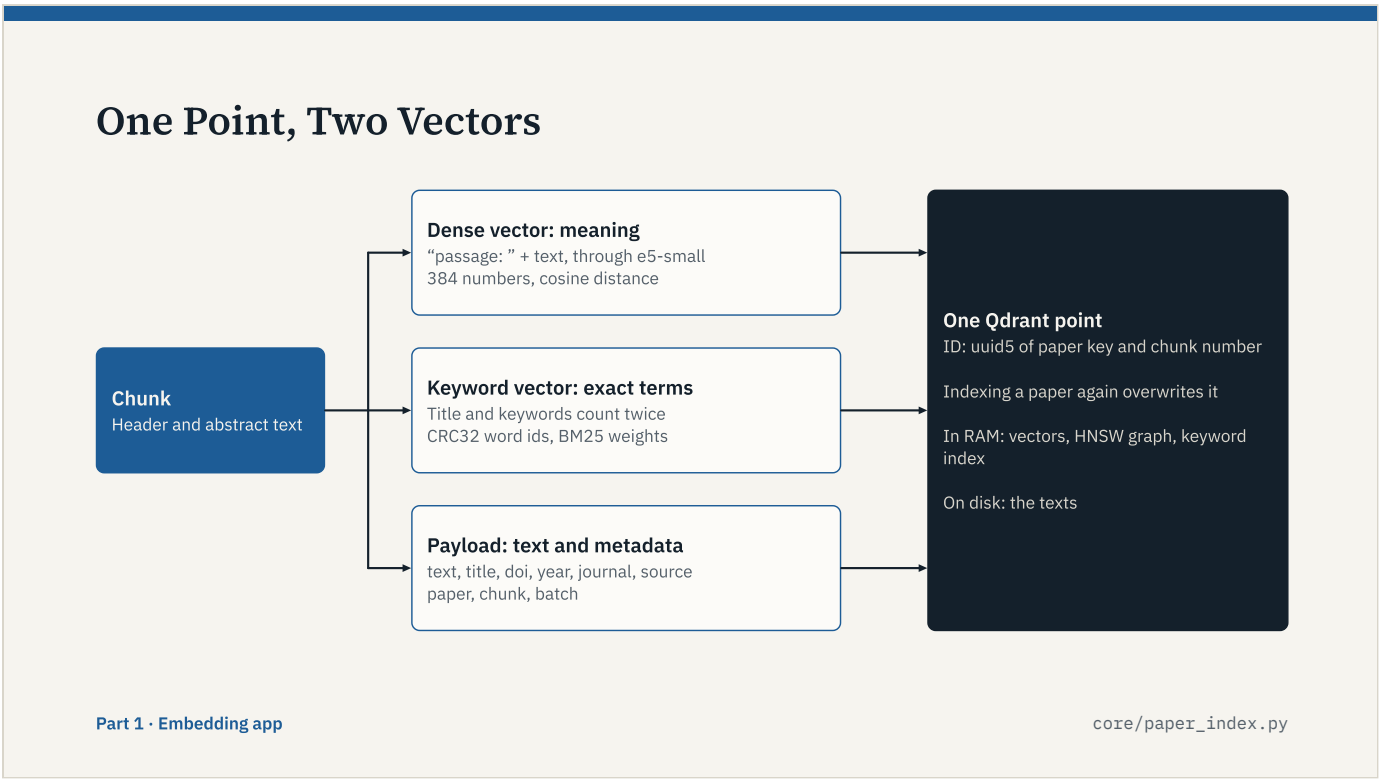
The left side shows the shape of a chunk. It starts with a header: title, keywords, authors, journal and year. Then comes the abstract.

Why repeat the header? Because a chunk must make sense on its own. When search finds a chunk, it already knows which paper it comes from.

The limit is 512 tokens. The code counts about 4 characters per token, so that is about 2,048 characters, including the header. Most abstracts fit in one chunk.

A long abstract is split at sentence boundaries, never inside a sentence, with about 80 tokens of overlap, so an idea at the boundary is not lost. Every part keeps the header.

On the full index, the average is about 1.3 chunks per paper.



SCRIPT

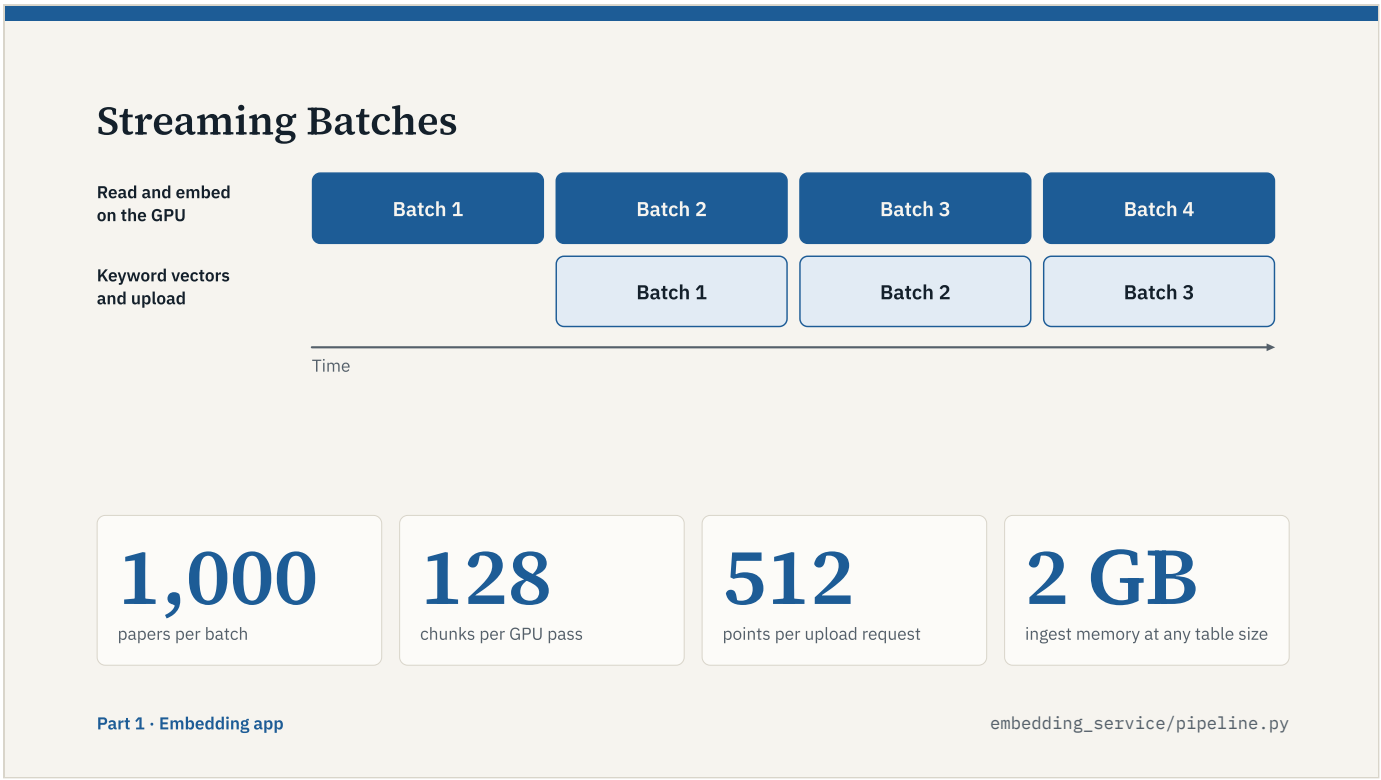
This slide is the heart of the index. Each chunk becomes one point in Qdrant, and each point has two vectors.

The first is the dense vector. The chunk goes through multilingual-e5-small with the prefix "passage:". The result is 384 numbers that capture meaning. This finds papers that use different words for the same idea.

The second is the keyword vector, a BM25 vector. We tokenize the text, remove stop words, and hash each word to a number with CRC32. The weights use the BM25 formula, and the title and keywords count twice. This finds exact terms, such as an acronym, that a dense vector can miss. Qdrant applies the IDF part itself, so word rarity stays correct when we add more papers.

The third part is the payload: the text and metadata.

The point ID is a stable hash of the paper key and the chunk number. So indexing the same paper again overwrites it, and does not duplicate it. Vectors and the keyword index stay in RAM, and the texts stay on disk.



SCRIPT

How does this scale to millions of papers without running out of memory? The answer is streaming.

Papers are handled in batches of 1,000. The reader fills a batch, and the GPU embeds it, 128 chunks at a time. The upload to Qdrant then runs in a background thread while the GPU starts on the next batch.

You can see that in the diagram. While batch 1 uploads, batch 2 is already on the GPU. So the GPU rarely waits. Keyword vectors are computed in that background step too.

At most two batches are in memory at any moment. That is why the ingest process stays at about 2 GB, whatever the size of the table.

One more detail: Qdrant builds its vector index, the HNSW graph, while papers are being uploaded. Building it only at the end turned out to be slower here, because the points pile up and get compacted again and again.

Two Ingest Modes

Aspect	Add to index (default)	Initialize index
Writes to	The live collection	A new collection, papers_<version>
Papers	New papers from this folder	This folder only
Duplicates	Skipped: same DOI, or same title without DOI	Skipped within the run
Goes live	As points are uploaded	At the end, by switching the alias
On failure or cancel	Points of this run are deleted by batch id	The new collection is dropped
Use it for	Each new data folder	A fresh start or a new index format

Part 1 · Embedding app `embedding_service/pipeline.py`

SCRIPT

There are two ways to run an ingest.

Add to index is the default. It writes into the current collection and skips papers that are already there. If there is no index yet, it simply builds one.

Initialize index starts over. It builds a completely new collection from this folder only. The current index is kept until the new one is complete, and only then does the alias switch.

The difference shows when something goes wrong. Every point carries a batch field with the run's version. So if an Add run fails or is cancelled, the app deletes exactly those points. If an Initialize run fails, the app drops the new collection, and the current index was never touched.

One honest exception. This cleanup needs the worker to stay alive. After a power cut during an Add run, the points already uploaded stay in the index.

Failure Handling

Event	Response
GPU out of memory	Halve the batch size; at size 8, continue on the CPU
Qdrant restarts	Wait up to 3 minutes, then carry on
MyISAM table marked crashed	Repair the staged copy and read again
Cancel pressed	Stop at the next check and roll back; under 10 s
Service restarts during a job	Job shown as interrupted; state saved in last_job.json
Ingest and export together	Second request refused; a snapshot must not change

Part 1 · Embedding app `embedding_service/jobs.py`

SCRIPT

A long job on a desktop PC will meet problems, so each known failure has a defined response.

If the GPU runs out of memory, the batch size is halved and the batch is retried. At batch size 8 it moves to the CPU.

If Qdrant restarts, the worker waits up to three minutes and carries on.

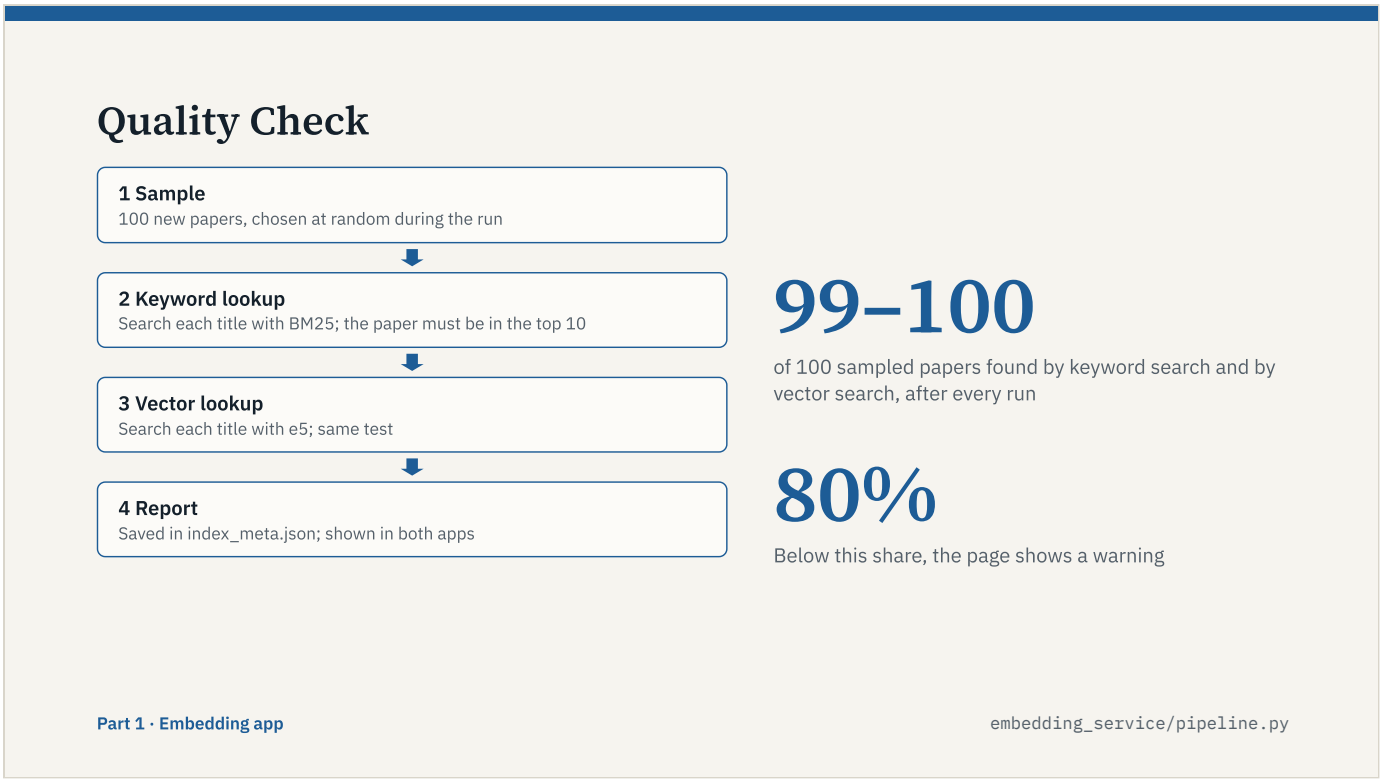
A crashed MyISAM table is repaired in the staged copy and read again.

If the user presses Cancel, the worker stops at its next check, and the rollback runs. We measured this at under 10 seconds.

If the service itself restarts during a job, the job is shown as interrupted, because its state is saved to a file.

And the app refuses to run an ingest and an export at the same time, because a snapshot must not change while it is written.

The result is that a failed night run does not leave you guessing. You can see what happened, and the index is still usable.



SCRIPT

Stage 6 answers one question: can we really find what we just indexed?

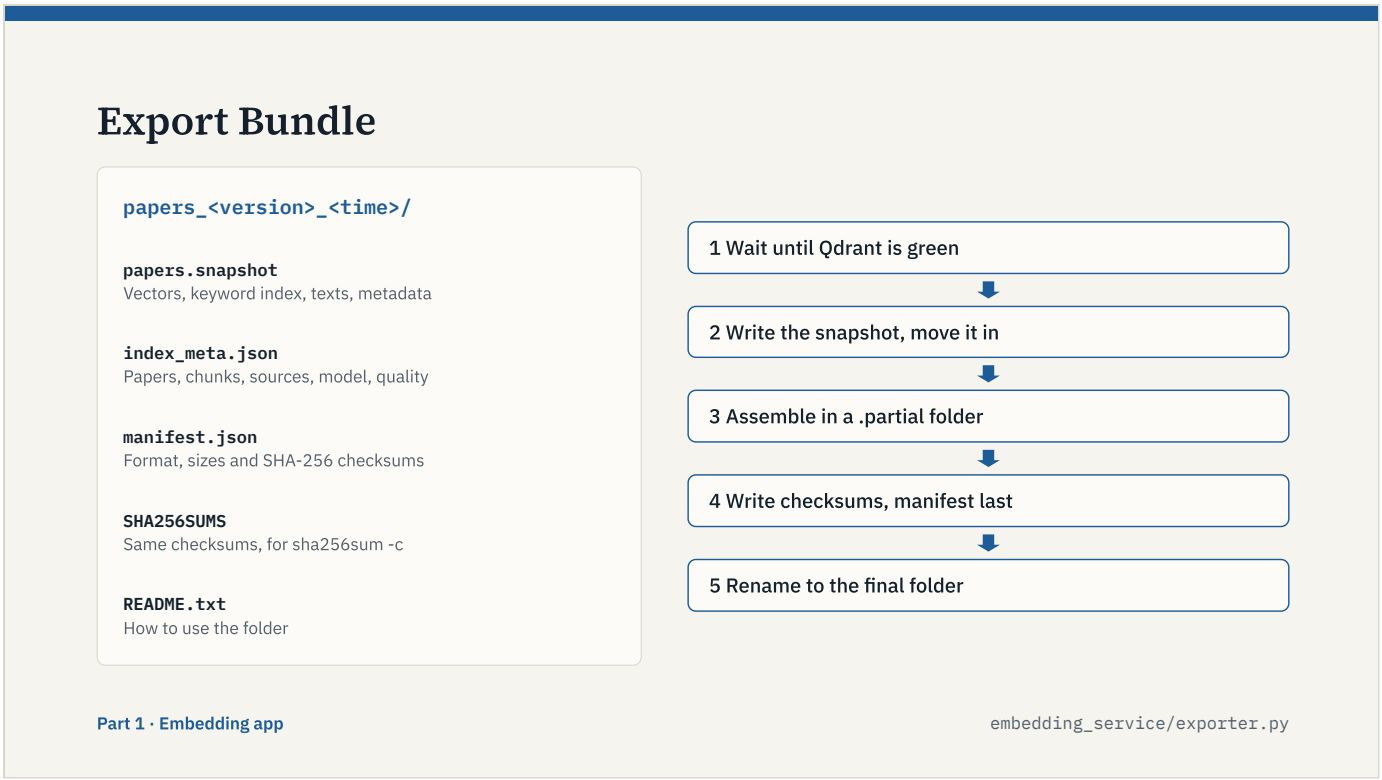
During the run, the worker keeps a random sample of 100 new papers. It uses reservoir sampling, so the sample is fair however large the run is.

After the upload, it searches for each paper by its title, twice: once with keyword search and once with vector search. The paper must appear in the top 10.

A low score points to a broken index, for example a wrong model or a tokenizer problem. Below 80 percent, the page shows a warning.

In our runs, 99 to 100 of the 100 papers were found by both searches. The result is saved in index_meta.json, so it travels with the bundle, and the search admin sees it too.

Please note what this test is. It proves the index works. It does not measure how good the final answers are.



SCRIPT

When the index is ready, one button exports it. The result is the one folder on the left.

papers.snapshot is the main file. It is a Qdrant snapshot with the vectors, the keyword index, the texts and the metadata. So the search PC rebuilds nothing. index_meta.json describes the index. manifest.json holds the format, the model, the sizes and the checksums. SHA256SUMS has the same checksums, so you can check a copy by hand.

On the right is how the export stays safe. It first waits until Qdrant reports green, so the snapshot contains the finished vector index. Qdrant writes the snapshot on the same disk, so it is moved, not copied.

Everything is assembled in a hidden folder ending in .partial. The manifest is written last. Then the folder is renamed in one step. So if you see a bundle folder with a manifest, you know it is complete. A restart cleans up any partial folder.

Embedding Performance

Step	Data	Time
GPU embedding alone	1,024 passages	452 passages/s
Initialize from 3 MySQL tables	127,883 papers, 149,181 chunks	7.8 min
Add a 2.8 GB MySQL table	945,611 papers, 1,219,887 chunks	81 min
Add a RIS file to the index	8,788 papers	5.7 min
Export the full index	1,082,282 papers, 5.9 GB bundle	6.3 min
Cancel a running ingest	Added papers removed	Under 10 s

Development PC: RTX 2060 6 GB, hard disk, WSL2 with 16 GB, 16 CPU threads

Part 1 · Embedding app `embedding/README.md`

SCRIPT

These are measured numbers from our development PC: an RTX 2060 with 6 GB, a hard disk, and WSL2 with 16 GB of memory. So this is modest hardware.

Embedding alone reaches about 450 passages per second. The whole pipeline reaches about 250 to 320 chunks per second, because Qdrant is indexing at the same time.

The first index, about 128 thousand papers, took under 8 minutes. Adding the big 2.8 gigabyte table took 81 minutes. Adding a small RIS file took 5.7 minutes, and most of that was reading the keys of the papers already indexed.

Exporting the full index of 1.08 million papers took 6.3 minutes and produced a 5.9 gigabyte bundle.

Embedding takes most of the time, so a faster GPU speeds up an ingest almost in proportion.

That completes the embedding app. We now have a tested bundle, and we can carry it to the search PC.

PART 2

The Search App

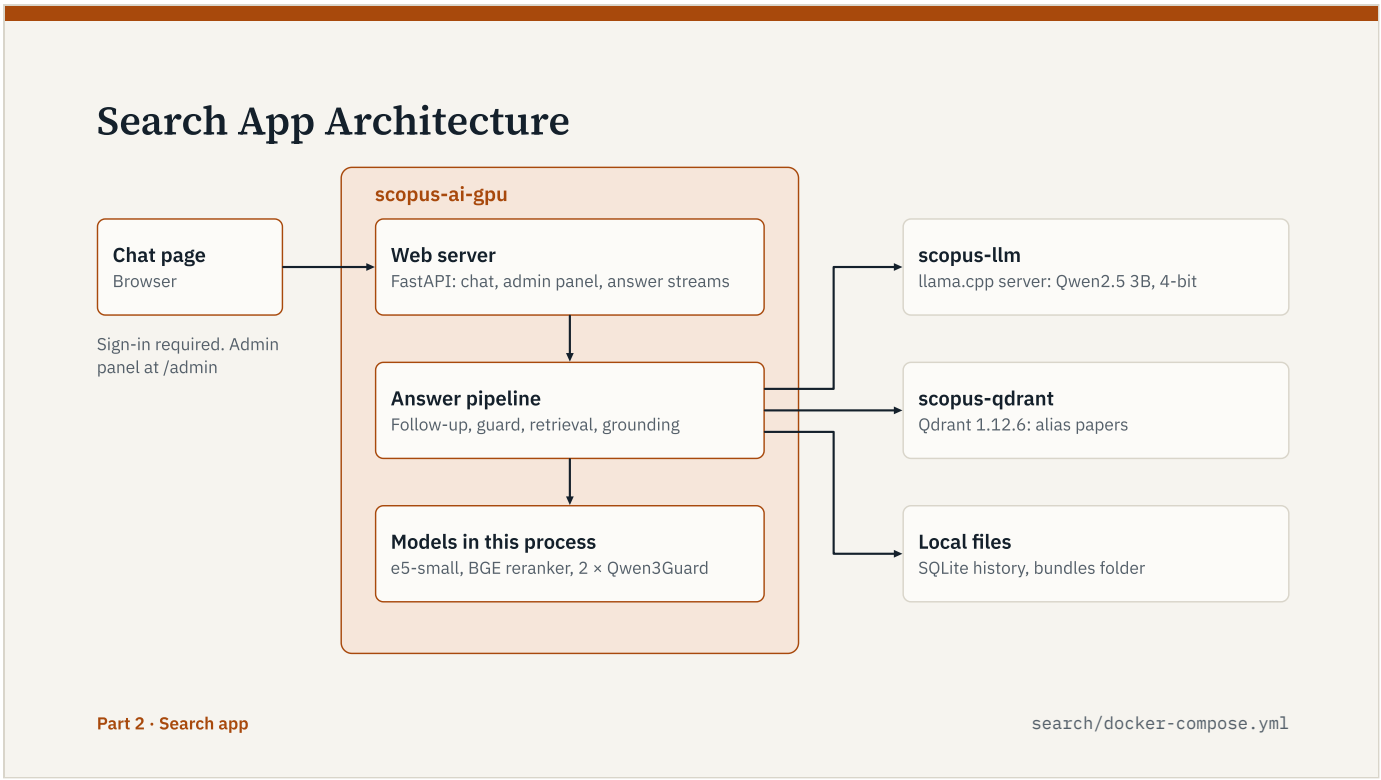
Imports the bundle and answers research questions only from the indexed papers.

SCRIPT

Now Part 2, the search app. We arrive with a bundle on a USB drive, and we leave with an answer on a screen.

The job has two halves: import the bundle safely, and answer questions.

Most of the engineering here serves one idea: the system must say only what the papers support. Keep that idea in mind, because each step I show is there to protect it.



SCRIPT

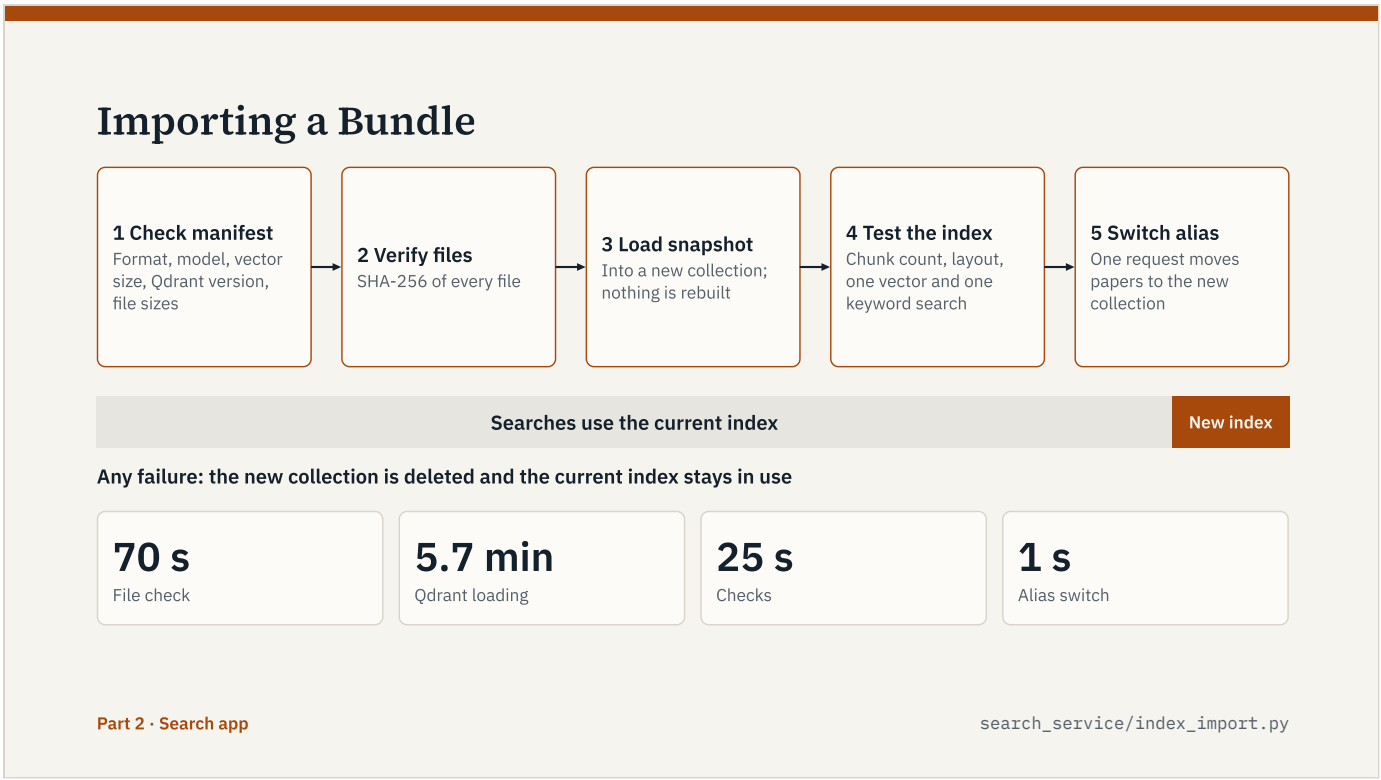
The structure mirrors the embedding app, which makes it easy to remember.

The orange box is scopus-ai, our Python code. The web server is FastAPI on port 7860. It serves the chat page and the admin panel, and it streams answers to the browser.

The answer pipeline coordinates everything for a question. Four small models run directly in this process: the embedding model for questions, the BGE reranker, and two Qwen3Guard safety models. On a 6 GB GPU, the question safety model moves to the CPU automatically.

The language model runs separately, in its own container: the llama.cpp server with Qwen2.5 3B, quantized to 4 bits. Keeping it separate is what lets us scale it: one copy of the weights, and several parallel slots.

Qdrant holds the imported index, behind an alias called papers. SQLite stores accounts, chats and the activity log.



SCRIPT

The first job is to load a bundle. The administrator copies the folder into bundles, opens the Index tab, and presses Import. Five things happen.

One: the manifest is checked. That means the index format, embedding model, vector size, Qdrant version and file sizes.

Two: the SHA-256 of each file is verified, so a damaged USB copy is caught here.

Three: Qdrant loads the snapshot into a new collection. Nothing is rebuilt.

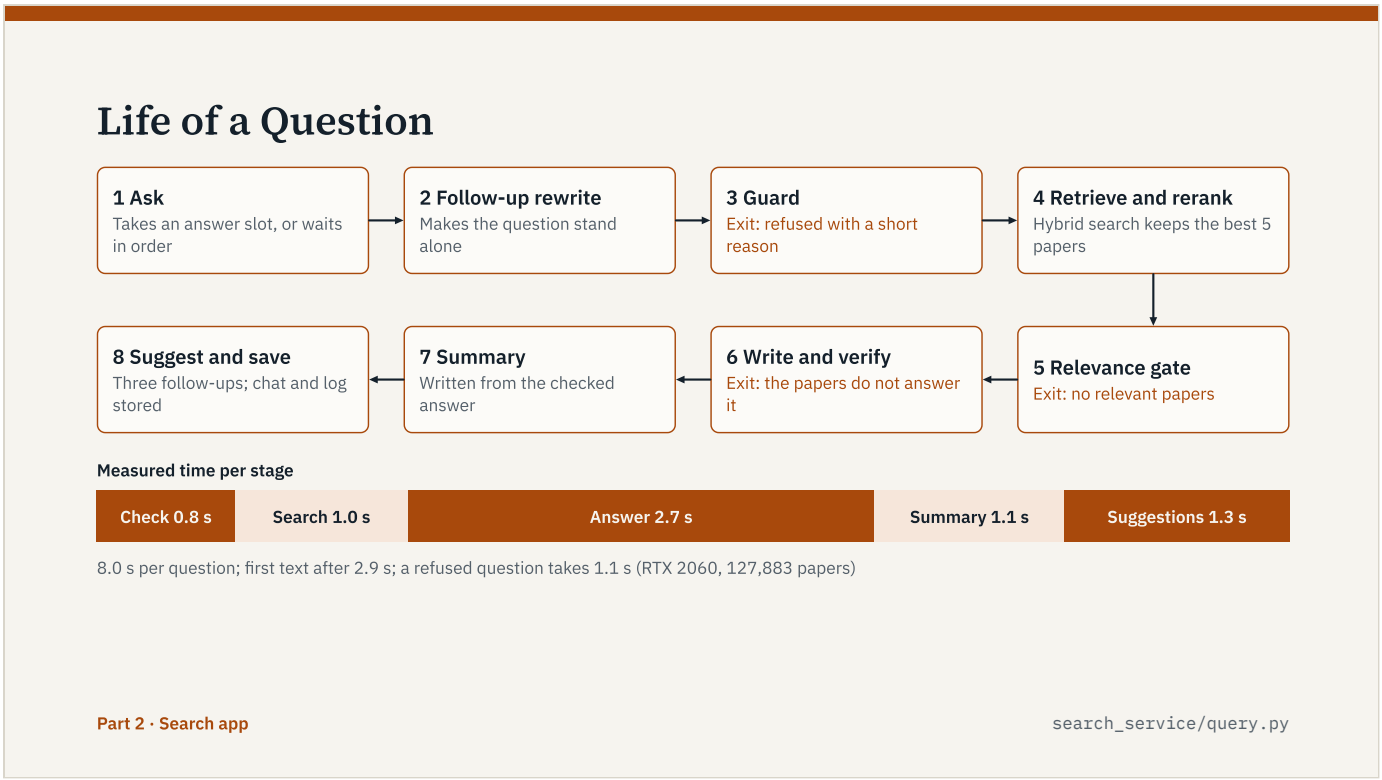
Four: the new collection is tested: the number of chunks, the vector layout, one vector search and one keyword search.

Five: the alias papers switches to the new collection, in a single request.

Look at the bar underneath. During steps 1 to 4, users keep searching the current index. At the switch they move to the new one, with no restart. If anything fails, the new collection is deleted and the current index was never touched.

The cost is memory, because Qdrant briefly holds both indexes. A low-memory option deletes the current index first, but then search is off during the import.

The numbers at the bottom are the measured import of 1.08 million papers: 7.8 minutes in total.



SCRIPT

Now the main workflow: what happens to one question. I will give the full path here, then zoom into each step.

Step 1: the question arrives and takes an answer slot. If all slots are busy, it waits in arrival order.

Step 2: if it is a follow-up, it is rewritten so it stands alone.

Step 3: the guard decides whether we answer at all.

Step 4: hybrid search finds candidate papers, and the reranker keeps the best five.

Step 5: if even the best paper scores too low, we stop and say so.

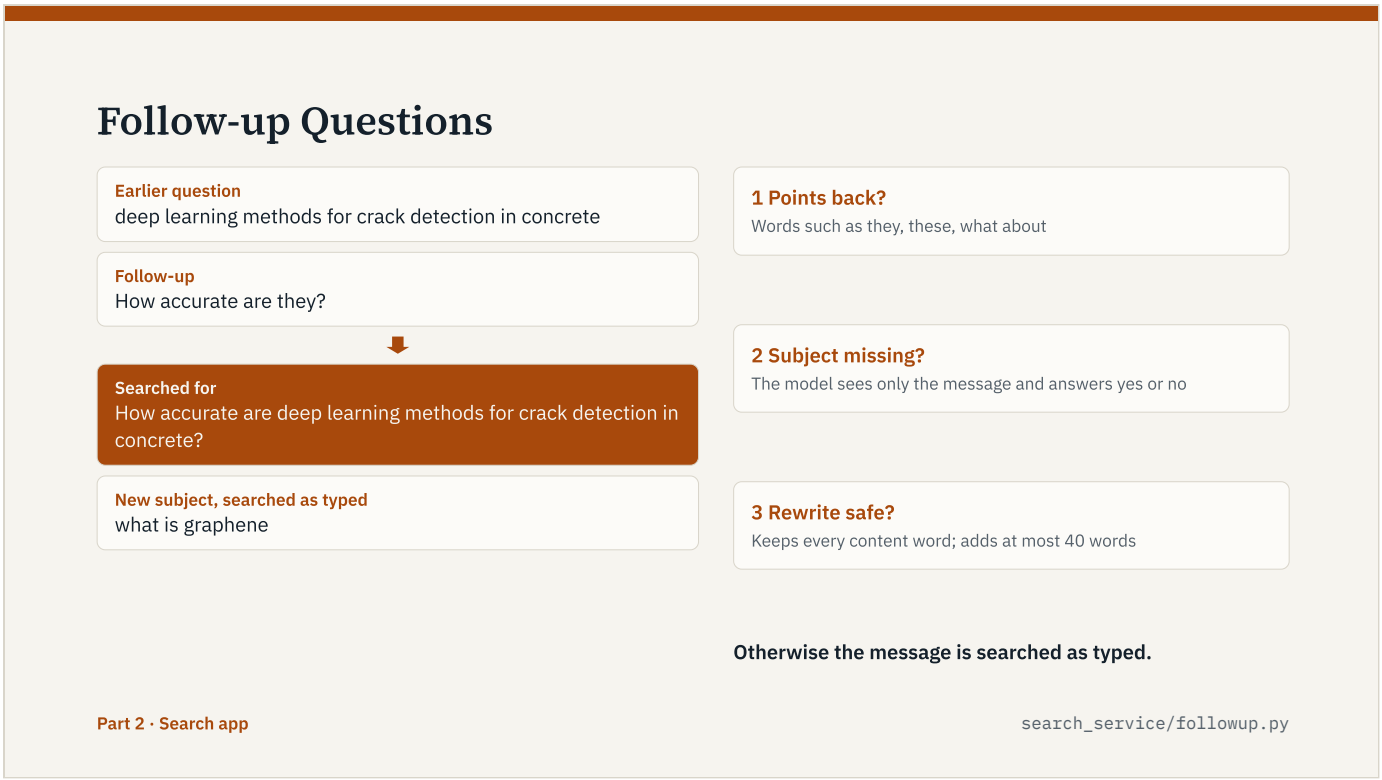
Step 6: the language model writes, and every sentence is checked while it streams.

Step 7: a summary is written from the checked answer.

Step 8: three follow-up questions are suggested, and the chat and the log are saved.

Notice the three exits. A question can stop at the guard, at the relevance gate, or when no sentence survives the check. At each exit the user gets a clear message instead of a weak answer.

The bar shows the measured time per stage. The whole question takes about 8 seconds, and the first text appears after about 3 seconds.



SCRIPT

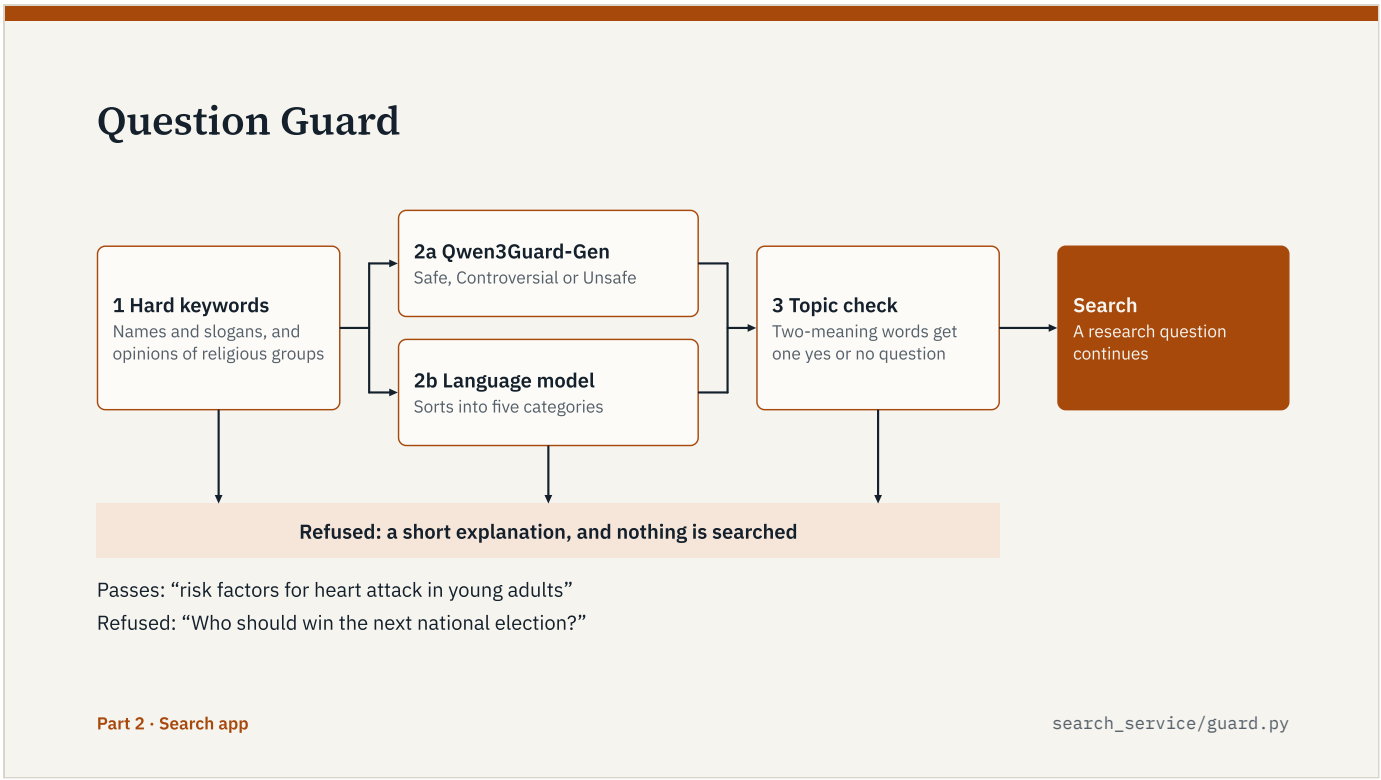
People talk in context. They ask: how accurate are they? A search engine cannot use that sentence.

So a follow-up is rewritten into a full question, using up to the last three turns of the chat. The rewritten text is shown to the user as "Searched for", so nothing is hidden.

The risk also goes the other way: an old topic must not take over a new question. If I ask about concrete, and then ask "what is graphene", the answer must be about graphene. So only real follow-ups are rewritten.

Three rules protect this. One: does the message point back, with words like they, these, or "what about"? Two: if not, the language model sees only the message, and answers yes or no: does it leave out its subject? Three: a rewrite is accepted only if it keeps every content word of the message and adds at most 40 words. Otherwise we search the message exactly as typed.

Questions that were refused are left out of this context.



SCRIPT

The guard decides which questions are answered. The aim is to judge what a question is about, and not only which words it contains.

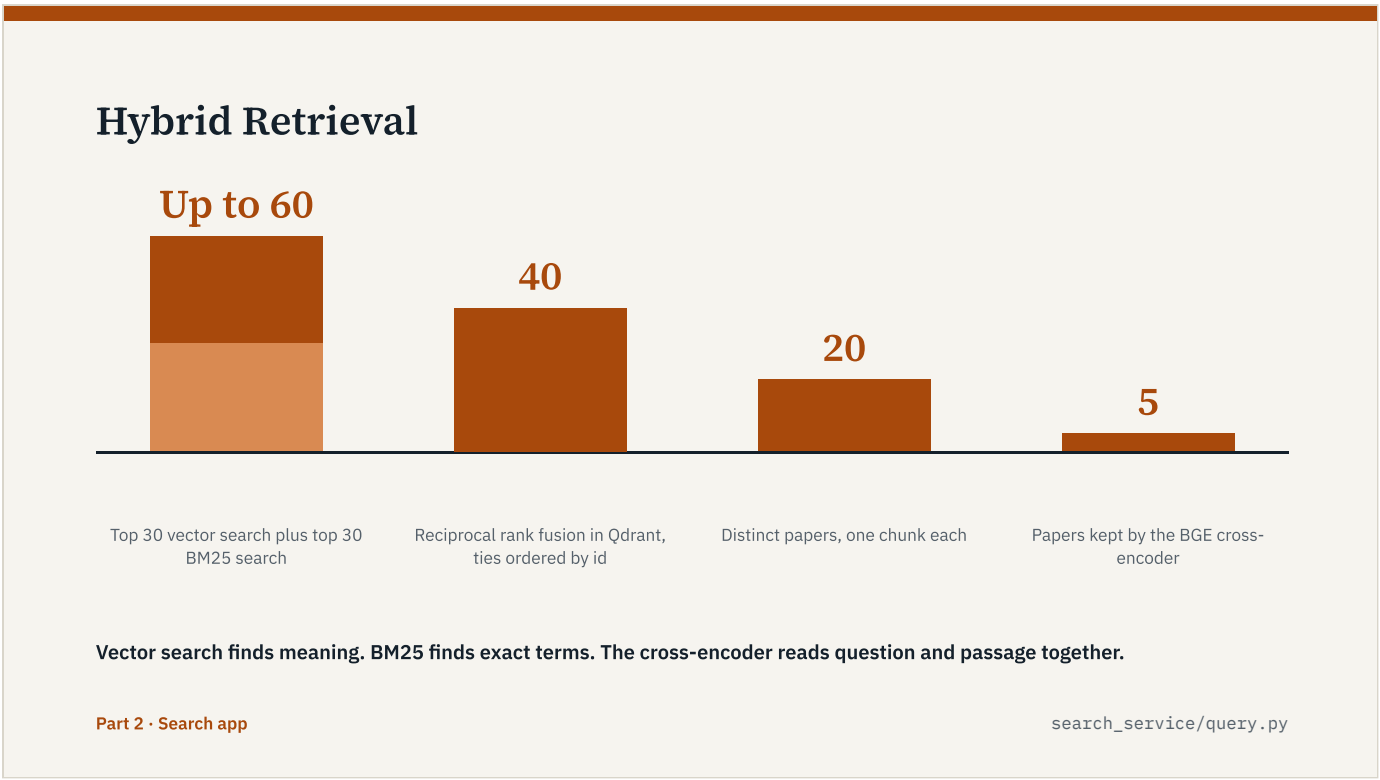
Layer 1 is a keyword list: names and slogans that practically never occur in a research question. These are always refused, and spelling variants are caught. Asking what a religious group thinks is also refused here.

Layer 2 runs two models at the same time. Qwen3Guard-Gen rates the question as safe, controversial or unsafe. The local language model classifies it into five categories: research, restricted, personal, harmful or off-topic. A grammar forces it to reply with exactly one of those words.

Layer 3 handles words with two meanings. "Attack" can mean war, but "heart attack" is medicine. For such a word, the model gets one narrow yes-or-no question, because small models are more reliable on yes or no.

For a follow-up, the guard also checks the message as typed, so a rewrite cannot carry a restricted question through.

A refusal is a short explanation, and nothing is searched. If the language model is down, the guard falls back to plain keyword blocking.



SCRIPT

Now retrieval. We use two searches, because each one is good at something different.

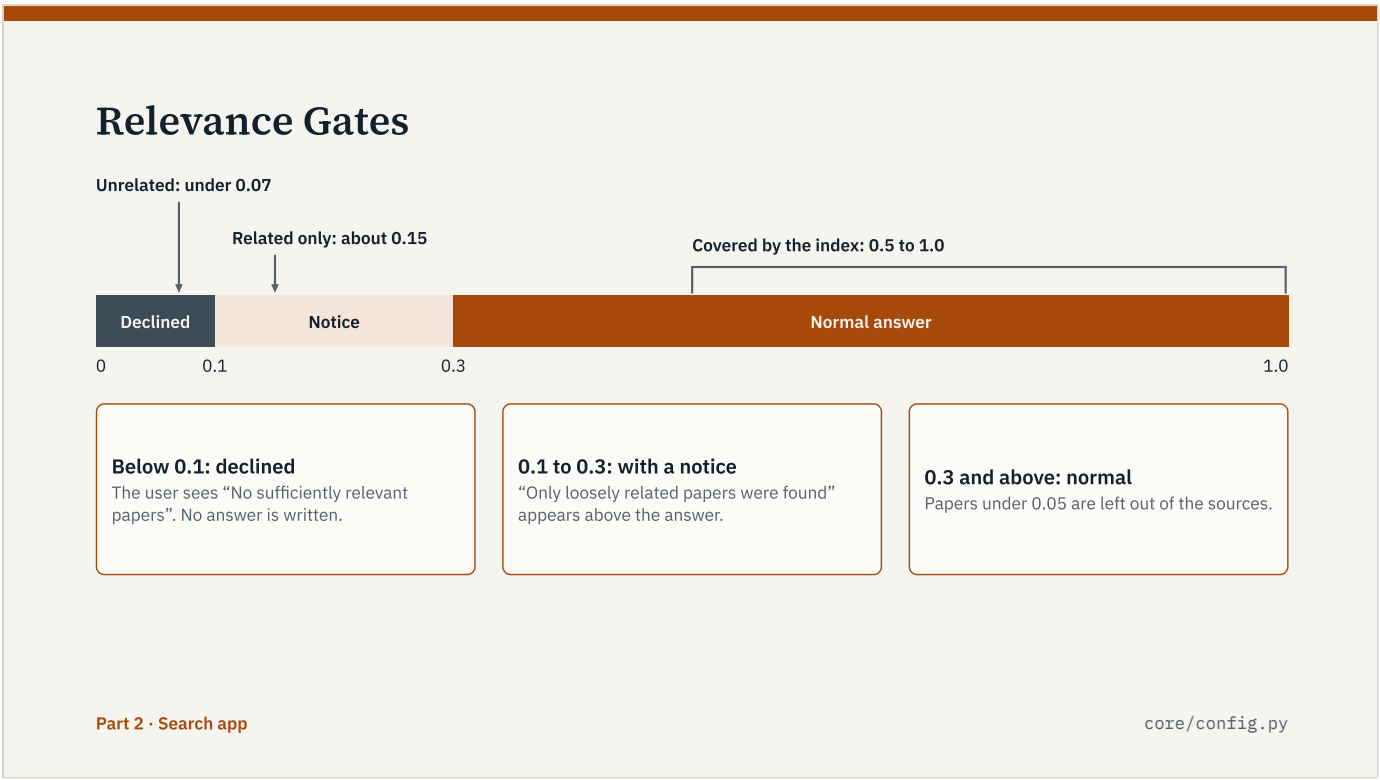
The question is embedded with the same e5 model, this time with the prefix "query:". That search finds meaning. The same question also becomes a BM25 query, which finds exact terms. Qdrant runs both, and takes the top 30 of each.

Reciprocal rank fusion then merges the two lists. It uses ranks, because vector scores and BM25 scores are on different scales. Fused scores often tie, and Qdrant orders ties arbitrarily. So the code takes a few extra candidates and sorts ties by ID. This means the same question always gives the same papers.

Several chunks of one paper can match, so we keep one chunk per paper, up to 20 papers.

Finally the BGE cross-encoder reads the question and each passage together. This is much more accurate than comparing vectors, but it is too slow to run on millions of chunks. It scores each paper from 0 to 1, and keeps the top 5.

That is the classic pattern: a fast search to find candidates, then a slower model to choose the best ones.



SCRIPT

The reranker score has a second use: it tells us whether the index covers the question at all.

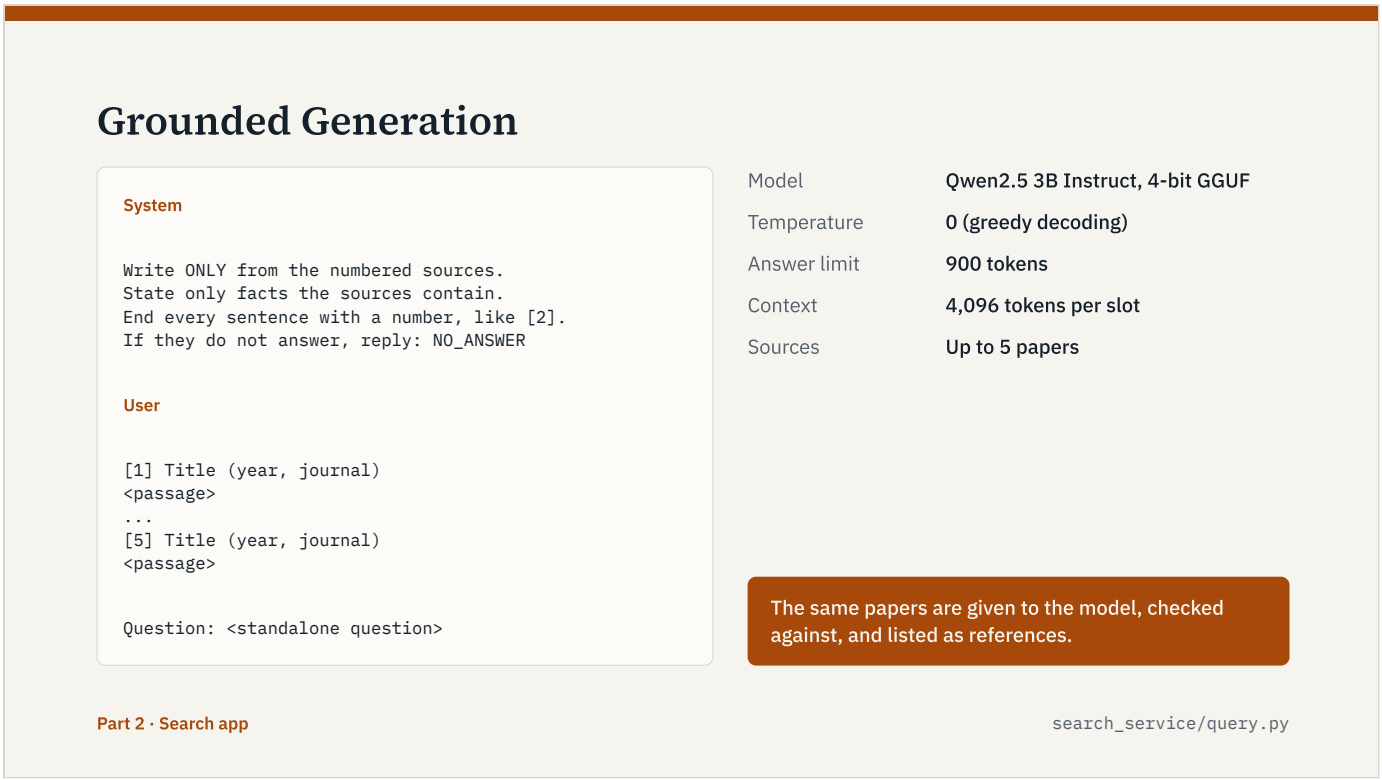
If the best paper scores below 0.1, we do not answer. The user sees "No sufficiently relevant papers". This stops the model from writing an answer out of unrelated sources.

Between 0.1 and 0.3 we do answer, but with a notice that only loosely related papers were found. From 0.3 upward, it is a normal answer.

These thresholds come from measurements, shown at the top of the scale. Unrelated questions score under 0.07. Topics with only related papers score around 0.15. Topics the index covers score between 0.5 and 1.

Papers scoring under 0.05 are left out of the sources, though we always keep at least the best one.

The two relevance thresholds are settings in the .env file, so they can be tuned without changing code.



SCRIPT

Now the answer itself. We give the model a tightly defined task.

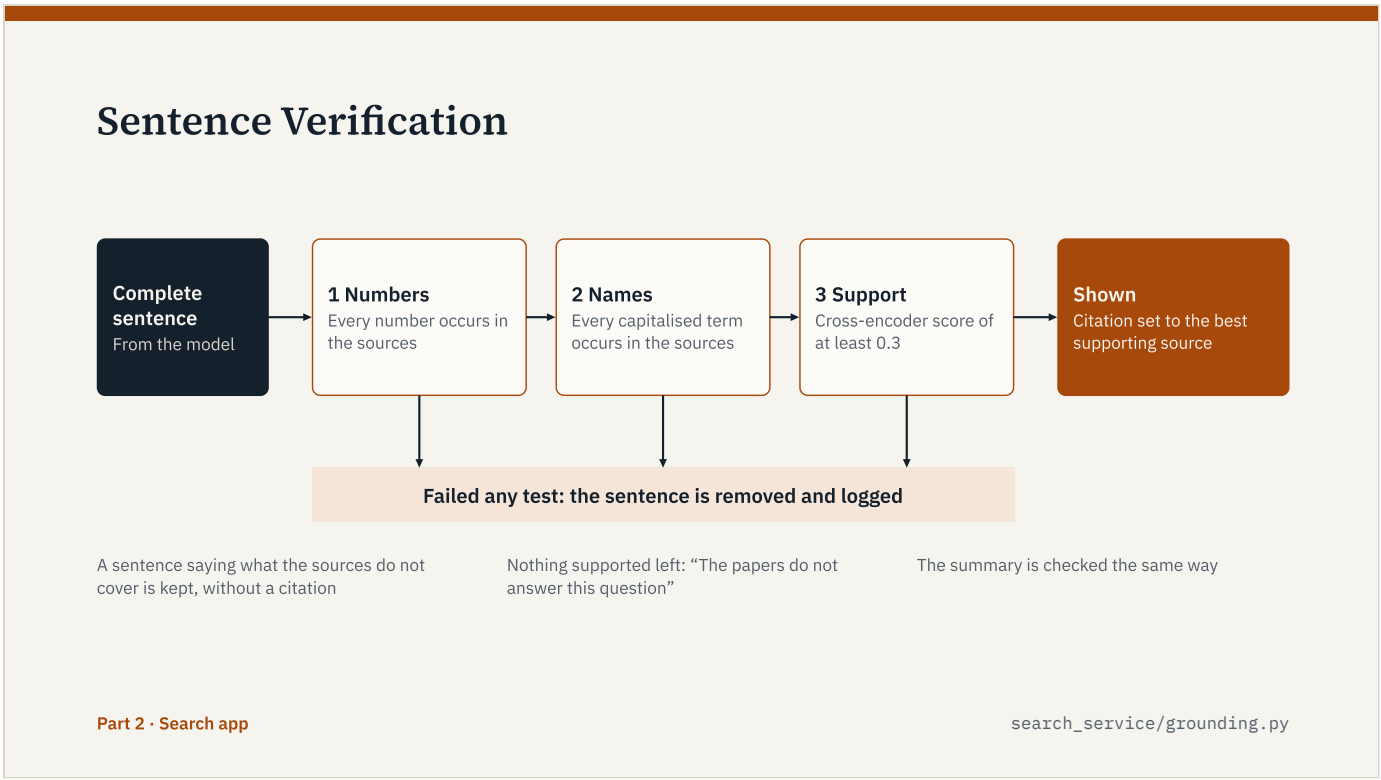
The system prompt says: write only from the numbered sources, and add no background knowledge. End every sentence with a source number. If the sources do not answer the question, reply with exactly NO_ANSWER.

After the rules come the sources: up to five papers, each with a number, title, year, journal and passage. Then the question.

Decoding is greedy, at temperature 0, which keeps the text close to the sources.

The same papers are used in three places. They are given to the model, the sentences are checked against them, and they are listed as references. So the references under an answer are exactly what it was written from.

A 3 billion parameter model is small, but here it has an easy job: restate what is in front of it. Still, instructions alone are not enough. A model can add outside knowledge anyway. That is why the next step exists.



SCRIPT

Every sentence must pass before the user sees it. There are three tests, cheapest first.

Test 1 is numbers. Every number in the sentence, such as a result, a year or a sample size, must occur in the sources.

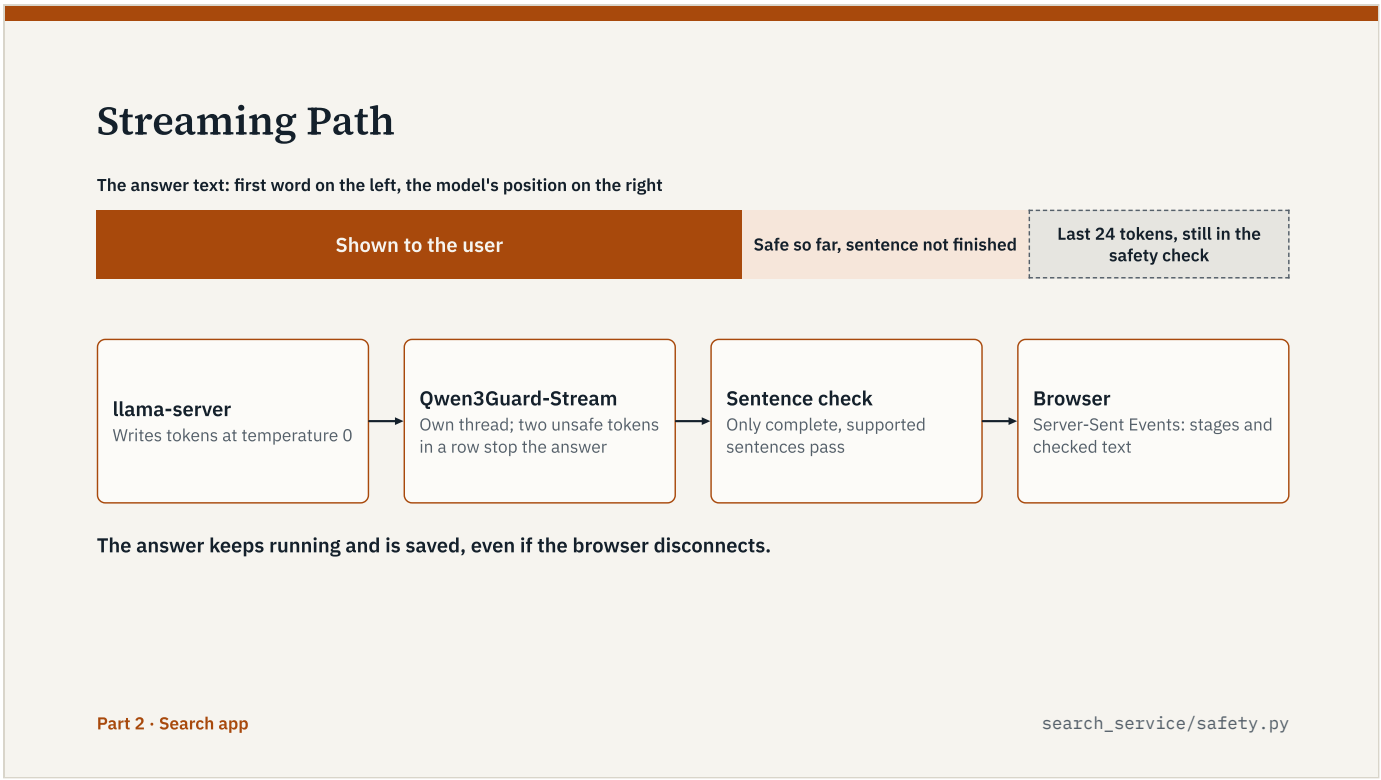
Test 2 is names. Every name, acronym or capitalised term must occur in the sources. This catches invented authors and methods.

Test 3 is support. The cross-encoder from the search scores the sentence against each source. The best score must reach 0.3. This is the same reranker model, reused as a checker, so it needs no extra GPU memory.

If a sentence passes, its citation is set to the source that supports it best, preferring the one the model cited. If it fails, it is removed and logged, and the admin panel counts it.

There is one exception. A sentence that only says what the sources do not cover is kept, without a citation, as long as it adds no new numbers.

If nothing supported survives, the user gets: "The papers found in the database do not answer this question". The summary is checked the same way.



SCRIPT

All of this happens while the answer streams, so the user does not wait for the full text.

Think of the answer as three zones, shown in the bar. The model writes at the far right. Qwen3Guard-Stream checks each token for safety, and text only counts as checked once 24 more tokens have been checked after it. Then the sentence check takes over, and only complete, supported sentences go to the browser. So the page shows text about one sentence behind the model.

If two tokens in a row are judged unsafe, generation stops at once, and the user sees a notice instead. The safety model runs in its own thread, so the writing does not wait for it.

The browser receives Server-Sent Events: stage messages, and the checked text so far. A waiting stream holds no worker thread, so many users can wait without slowing the site.

And the answer keeps running even if the browser disconnects. It is saved in the chat, so the user finds it when they return.

What the User Receives

Searched for
Shown when a follow-up was rewritten

Summary
Two to four sentences, also checked

Detailed answer
Paragraphs or lists, with citations [n]

References
Title, journal, year, DOI link, relevance

Related questions
Up to three clickable suggestions

Status	Meaning
answered	Full answer, as shown on the left
blocked	Guard or safety check stopped it
no_results	The papers do not cover the question
no_index	No bundle has been imported yet
unavailable	Language model server is down

Every chat is saved on the server. Users can search it, rename it or export it as Markdown.

Part 2 · Search app

`search_service/query.py`

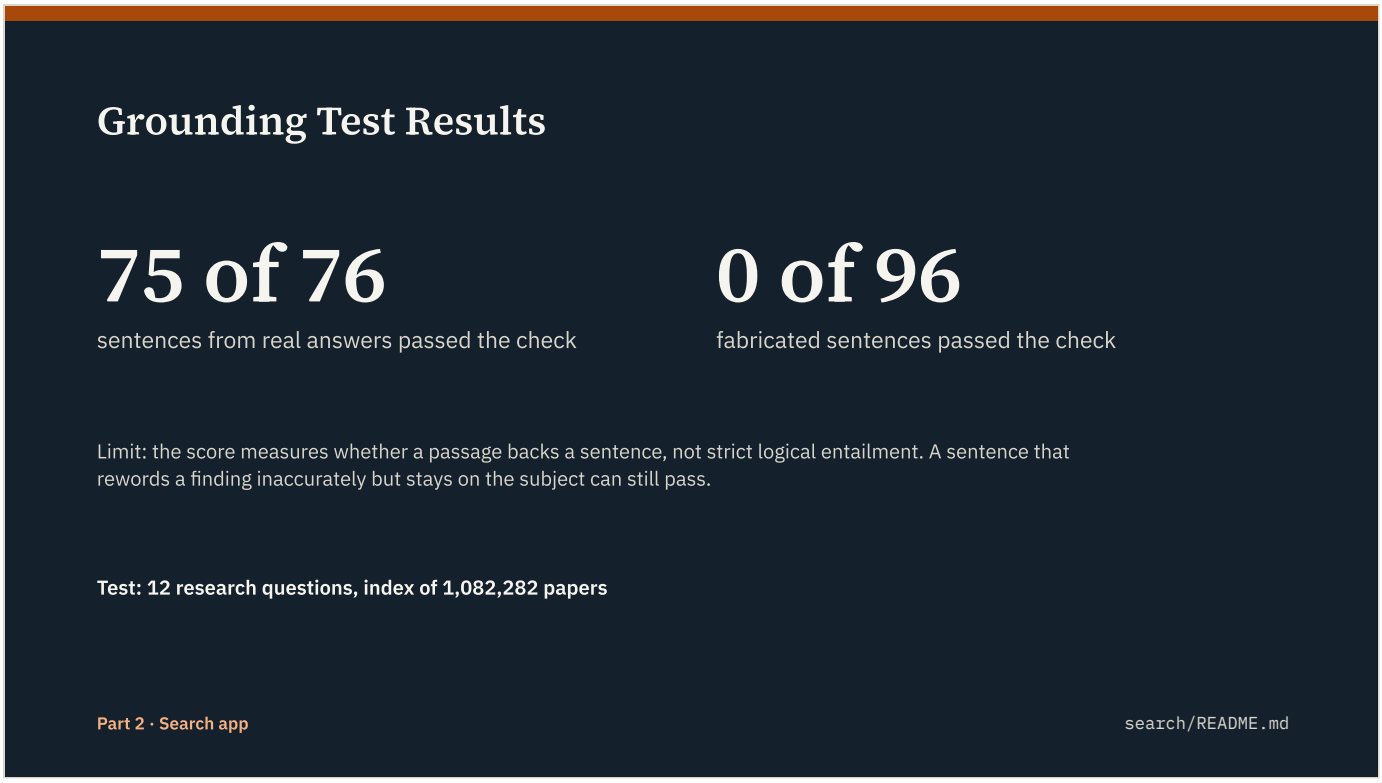
SCRIPT

This is what the user finally receives.

At the top is a short summary. Under it is the detailed answer, with clickable citations. Then come reference cards, with the journal, year, DOI link and relevance score. Last are three suggested questions to click. A "Searched for" line appears when a follow-up was rewritten, so the user can see what was really searched.

The table on the right lists all the outcomes, because a good system must also fail clearly. Answered is the normal case. Blocked means the guard or the safety check stopped it. No results means the papers do not cover the question. No index means no bundle has been imported yet. Unavailable means the language model server is down.

Each chat is saved on the server. Users can search it, rename it, or export it as Markdown.



SCRIPT

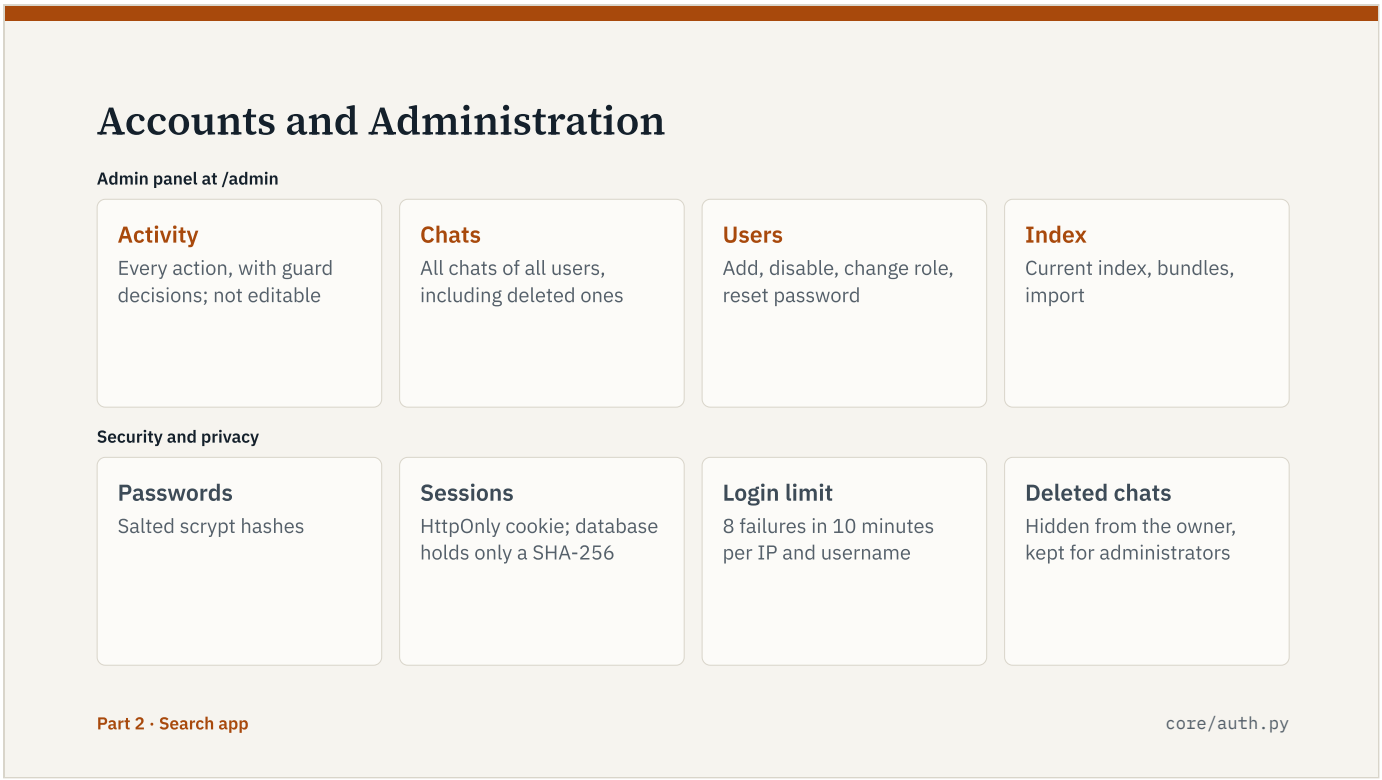
How do we know the sentence check works? We tested it on 12 research questions, on the index of about 1.08 million papers.

Of 76 sentences from real answers, 75 passed. So it almost never removes good content.

Then we tried 96 fabricated sentences: invented numbers and names, off-topic facts, and plausible general knowledge that the papers do not state. None of the 96 passed. In the end-to-end test, hallucinated sentences injected into the model's reply never reached the screen.

There is a limit, and I want to be direct about it. The support score measures whether a passage backs a sentence. It does not prove strict logical entailment. A sentence that rewords a finding inaccurately, but stays on the subject, can still pass.

The threshold can be raised, at the cost of dropping more sentences. And 12 questions is a useful check, but it is a small test, not a full evaluation.



SCRIPT

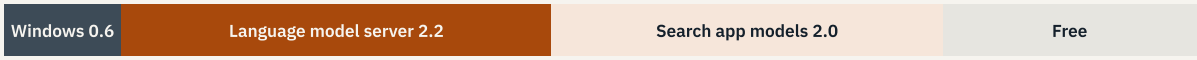
Now the operational side. Users need an account, and the first account becomes the administrator.

The top row is the admin panel, which has four tabs. Activity logs everything: logins, questions with the guard decision, exports and imports. It cannot be edited or deleted from the page. Chats shows all chats of all users. Users is for adding, disabling and resetting accounts. Index is the import screen we saw earlier.

The bottom row is security. Passwords are stored as salted scrypt hashes. A login creates a random token. It sits in an HttpOnly cookie, and the database keeps only its SHA-256 hash. After 8 failed logins in 10 minutes, that IP and username pair is locked out for a while.

Please note the last card. Users see only their own chats. But when a user deletes a chat, it is only hidden from them. Administrators can still read it. If this system serves a wider group, users should be told that.

Capacity on a 6 GB GPU



GPU memory in GB on a 6 GB card; 5.1 of 6.1 GB in use under load

Measure	Result
One question	8.0 s
Throughput	490–585 answers per hour
16 users at the same moment	All in 115 s, median 57 s
10 users, one question a minute	Mean 13.5 s, p95 25.6 s

LLM_PARALLEL	Users
1	24–28
2	37
4	41

2 minutes of reading between questions, mean response at most 60 s

Part 2 · Search app

search/README.md

SCRIPT

How many users can this serve? These are load test results on the same RTX 2060, on an index of 127,883 papers, with all checks switched on.

The bar shows how the 6 gigabyte card is used. About 5.1 gigabytes are in use under load.

One question takes about 8 seconds. Throughput is about 490 to 585 answers an hour. When 16 users asked at the same moment, all were answered within 115 seconds, with a median of 57 seconds and no errors. With 10 users asking about once a minute, the mean response was 13.5 seconds.

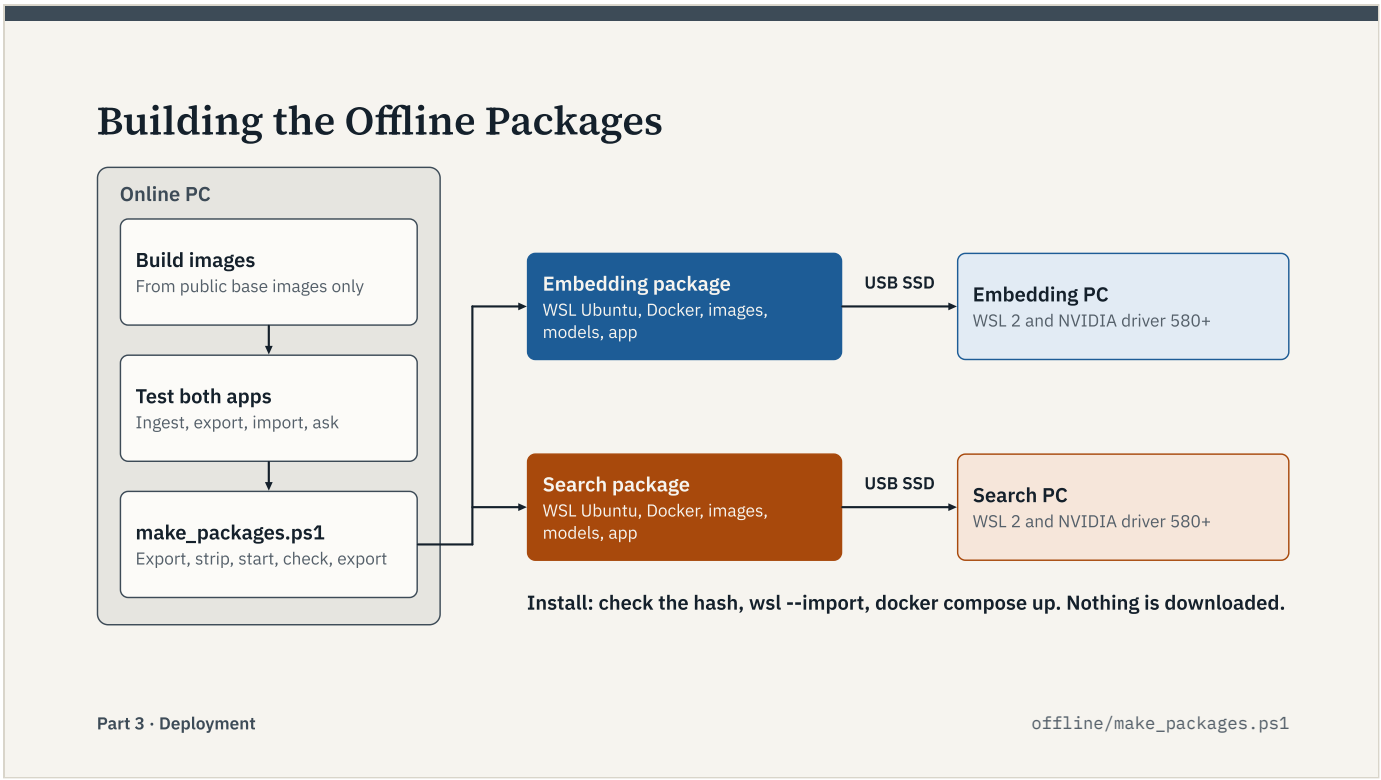
The table on the right estimates users. If people spend 2 minutes reading between questions, and we accept a mean response of 60 seconds, that is 24 to 28 users with one slot, 37 with two, and 41 with four. The gain is small on this GPU, because most GPU time goes to reading the prompts. A faster GPU improves both.

To scale, we change settings: raise LLM_PARALLEL, or point LLM_MODEL to a larger model. No code changes. And only questions use the GPU, so idle users cost nothing.



SCRIPT

Part 3 is short. Both apps must run on PCs with no internet. So how do they get there, and how does the index stay up to date?



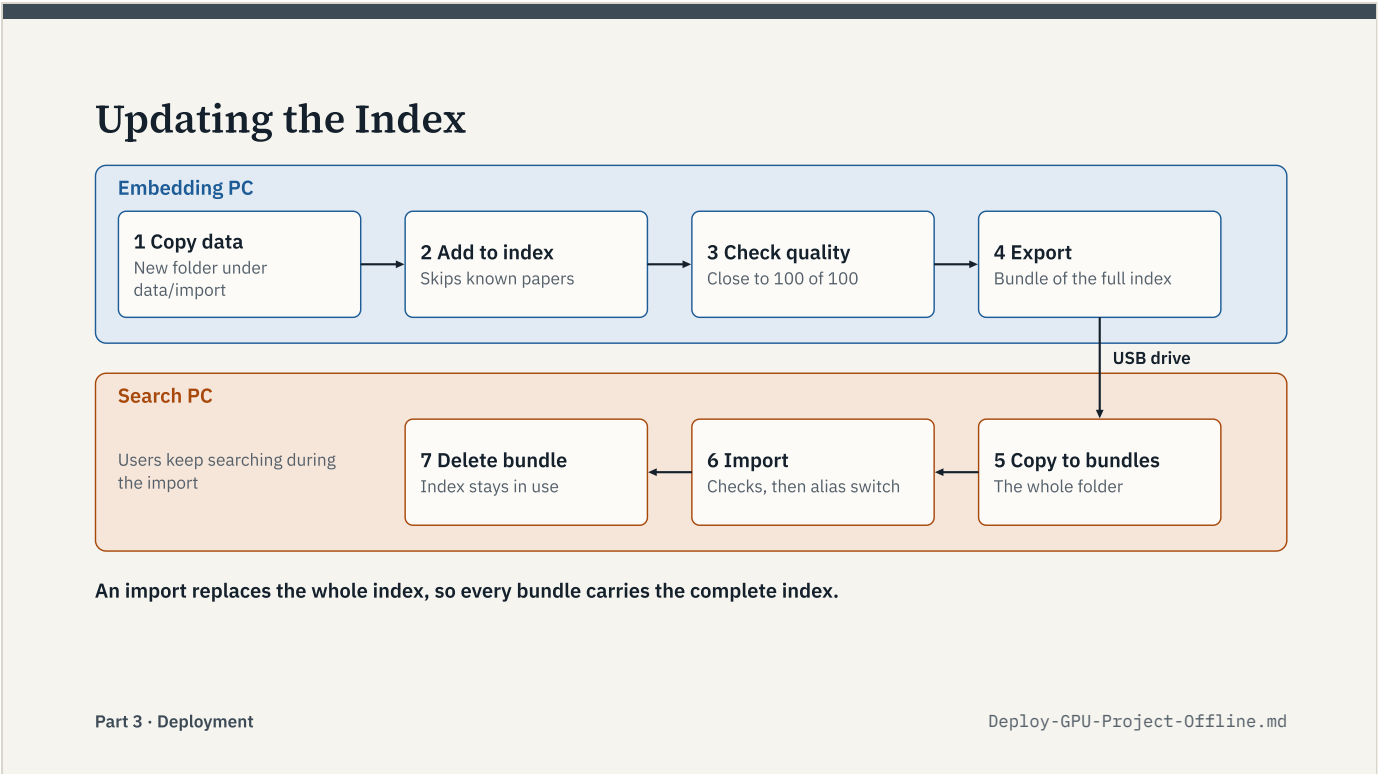
SCRIPT

Everything is built and tested on one online PC, in a WSL Ubuntu distribution that holds both apps. All images build from public base images. The search Dockerfile makes the search image and the llama.cpp image, and the embedding image is made from the search image.

Then one PowerShell script, make_packages.ps1, does the rest. It exports the distribution once. It imports a temporary copy, removes the search app, starts the embedding app, checks it, and exports the embedding package. Then it does the same the other way round for the search package. This takes 1 to 2 hours.

Each result is a plain .tar file with a SHA-256 file beside it. A package is a complete WSL Ubuntu with Docker, the NVIDIA toolkit, the images, the models and one app.

The offline PC needs only WSL 2 and the NVIDIA Windows driver, version 580 or newer. Installation is short: check the hash, run wsl --import, and run docker compose up. Nothing is downloaded there.



SCRIPT

Here is the regular cycle for adding papers.

Steps 1 to 4 are on the embedding PC, in blue. Copy the new data set into its own folder. Run Add to index. Check the quality line: both numbers should be close to 100 of 100. Then press Export.

The bundle crosses on a USB drive.

Steps 5 to 7 are on the search PC, in orange. Copy the folder into bundles, import it in the admin panel, and delete the bundle folder afterwards.

One thing to remember: an import replaces the whole index on the search PC. So each bundle must contain the complete index, not only the new papers. That is why we always add on the embedding PC and export everything.

Users are not interrupted at any point, unless we choose the low-memory option.

Known Limits

Limit	What to do about it
Index holds abstracts, not full text	Answers cover what abstracts state
Support score is not strict entailment	Raise GROUNDING_MIN_SCORE for stricter answers
A 3B model is small	Point LLM_MODEL to a larger GGUF on a larger GPU
Each bundle carries the full index	About 5.5 GB per million papers, each time
Import holds two indexes	Low-memory option: no search during the import
Hard keywords can refuse real research	Edit the lists in filters.py
Qdrant port 6333 is open, no password	Close it on an untrusted network

Part 3 · Deployment

SCRIPT

A fair picture needs the limits too.

The index holds titles, metadata and abstracts, not full text. So the system can only answer what abstracts say.

The sentence check is strong, but as I said, it is not strict entailment.

The 3B model is small. A larger model is a settings change on a larger GPU.

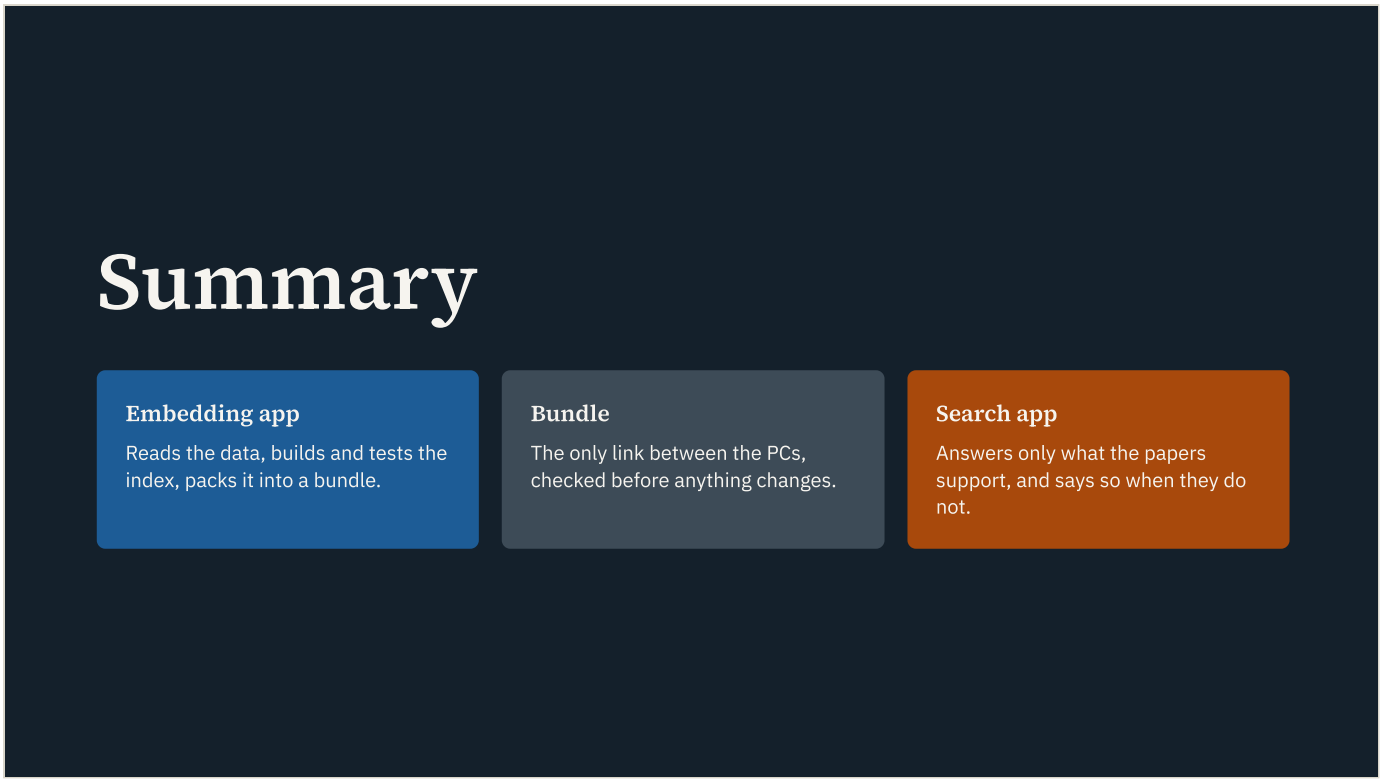
Bundles carry the full index, so there are no small updates. A million papers is about 5.5 gigabytes each time.

An import needs memory and disk for two indexes, unless we use the low-memory option.

The keyword list can refuse a genuine research question, because a word like "president" is always blocked. We can edit that list in filters.py.

Last is security on the search PC. Qdrant's port 6333 is published, and it has no password. That is acceptable on a trusted PC, but on an untrusted network it should be closed in docker-compose.yml.

None of these is a blocker, but you should know them before you rely on the system.



SCRIPT

Let me finish with three points.

First, the embedding app does the heavy work once. It reads the data, builds the index, tests it, and packs it into a bundle.

Second, the bundle is the only link between the two PCs. It is checked with checksums, format and model before anything changes.

Third, the search app answers only what the papers support. The guard, the relevance gate, the safety model and the sentence check can each stop an answer. And when the papers do not answer a question, the system says so.

Thank you for listening. I am happy to take your questions.

Prepared Answers

Question	Short answer
Why two PCs?	A long ingest never competes with users for the GPU
Why keywords and vectors?	Vectors find meaning; BM25 finds exact terms
Can a hallucination get through?	Invented numbers and names are caught; a subtle rewording can pass
Does it need a GPU?	Embedding can use the CPU, slowly; the LLM should stay on the GPU
What if a USB copy is damaged?	SHA-256 fails, import stops, current index stays
How are new papers added?	Add to index, export everything, import
Are chats private?	Users see only their own; administrators can read all

Appendix

SCRIPT

This slide is a backup for the question round. Keep it ready, and show it only if it helps. Here are fuller answers.

Why two PCs? An ingest of a million papers takes over an hour on the GPU. On a separate PC it never competes with users, and the search PC's GPU serves questions only.

Why both keywords and vectors? Vectors find the same idea in different words. BM25 finds exact terms like acronyms. Fusion gives us both, and both live in one Qdrant collection, so one snapshot moves everything.

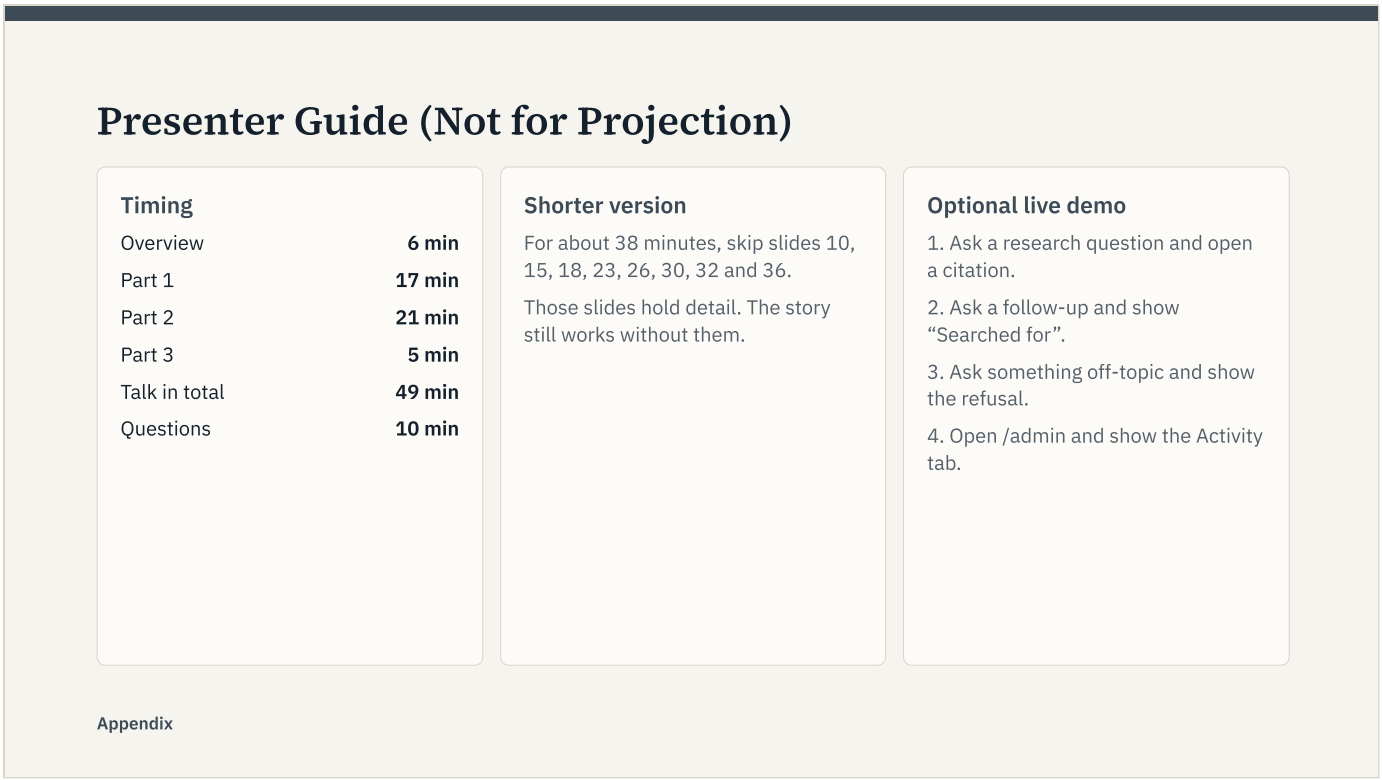
Can a hallucination get through? Invented numbers and names are caught. A subtle rewording that stays on the subject can pass, because the score is not strict entailment.

Does it need a GPU? The embedding app can run on the CPU, slowly. For the language model, an earlier test on the CPU took 72 seconds per answer instead of 9, so keep it on the GPU.

What if a USB copy is damaged? The SHA-256 check fails, the import stops, and the current index stays in use.

How are new papers added? Add to index on the embedding PC, export the complete index, and import it on the search PC.

Are chats private? Users see only their own chats. Administrators can read all chats, including deleted ones.



SCRIPT

This slide is for you, the presenter. It is not meant for projection.

Every slide has a full script in its speaker notes. Read the notes aloud a few times before the day, and then speak in your own words. On the diagrams, point at each box as you name it.

The timing assumes a calm speaking pace of about 130 words per minute. Practice once with a timer and adjust.

Colour helps the audience: blue always means the embedding app, orange always means the search app.

If you run a live demo, test it first with the apps already running and the models loaded, because the first question after a start can be slow.